

QEvict: Recoverable Quantized KV Eviction for Attention-Drift-Robust Long-Context Decoding

Ayushman Garg^{1,*}, Akshita Gupta^{1,*}, Shaswata Bhattacharya^{1,*}, Abhishek Gupta^{2,*}, Sandeep Kumar^{2,3}, Manoj Kumar¹

¹Mehta Family School of Data Science and Artificial Intelligence, Indian Institute of Technology Roorkee

²Yardi School of Artificial Intelligence, Indian Institute of Technology Delhi

³Department of Electrical Engineering, Indian Institute of Technology Delhi

Abstract

Autoregressive large language model inference is increasingly constrained by the memory footprint of the Key-Value (KV) cache. A dominant line of work reduces this footprint by evicting tokens that appear unimportant under attention-derived scores. However, such policies make an implicit irreversible decision: once a token is evicted, it cannot become useful again. We show that this assumption is brittle during decoding. Token and window importance drift as generated queries evolve, causing standard eviction policies to permanently discard states that later receive substantial attention under the full-cache model. To characterize this behaviour, we introduce *Future Missed Mass* and *Global LIR*, two diagnostics that measure future attention assigned to discarded states and the reactivation of historically inactive regions. We propose QEvict, a three-tier KV-cache management scheme that replaces binary retain-or-delete eviction with *recoverable eviction*. QEvict maintains high-confidence windows in full precision, stores intermediate windows in a quantized recoverable tier, and deletes only the lowest-confidence windows. During decoding, cumulative attention scores update window importance and when a quantized window becomes important again, it is dequantized and promoted to the full-precision. Under a fixed memory budget, this design preserves broader historical context while retaining exact full precision for the most important regions. Across long-context understanding, retrieval, and reasoning benchmarks, QEvict consistently improves over representative eviction and quantization baselines, reducing missed attention and improving information retention.

1 Introduction

Transformer-based large language models (LLMs) have achieved remarkable success across language understanding, generation, and reasoning (Vaswani et al. 2017; Brown et al. 2020; Touvron et al. 2023). Their deployment in long-context settings, however, is increasingly constrained by the key-value (KV) cache (Pope et al. 2023). Because the cache grows linearly with sequence length and batch size, its memory footprint can rival or exceed that of the model parameters at long context lengths. This limits batch capacity, increases inference latency, and raises serving costs. Hardware-aware attention kernels improve computational efficiency (Dao 2023), while memory-management systems reduce allocation over-

Table 1: Comparison of QEvict with representative KV-cache compression methods. A ✓ indicates the presence of a desirable property, while ✗ indicates its absence.

Method	Selective Eviction	Low-Bit Retention	Window Routing	Dynamic Recovery
StreamingLLM (Xiao et al. 2024)	✓	✗	✗	✗
SnapKV (Li et al. 2024)	✓	✗	✗	✗
AdaKV (Feng et al. 2024)	✓	✗	✗	✗
CriticalKV (Feng et al. 2025)	✓	✗	✗	✗
DefensiveKV (Feng et al. 2026)	✓	✗	✗	✗
KIVI (Liu et al. 2024)	✗	✓	✗	✗
KVQuant (Hooper et al. 2024)	✗	✓	✗	✗
ZipCache (He et al. 2024)	✗	✓	✗	✗
QEvict (Ours)	✓	✓	✓	✓

head (Kwon et al. 2023); neither changes the fundamental storage complexity of the KV cache.

A substantial body of work therefore compresses the KV cache through eviction. StreamingLLM (Xiao et al. 2024) retains a small set of attention sinks together with recent tokens, while H₂O (Zhang et al. 2023) exploits attention sparsity to preserve accumulated heavy hitters. Subsequent methods refine the selection process through prompt-time observation (Li et al. 2024), adaptive budget allocation across attention heads (Feng et al. 2024), value-aware importance estimation (Feng et al. 2025), and robustness to uncertain future attention (Feng et al. 2026). In parallel, quantization methods such as KIVI (Liu et al. 2024), KVQuant (Hooper et al. 2024), and ZipCache (He et al. 2024) retain broader historical coverage by representing KV states at reduced precision.

Despite this progress, prevailing eviction methods retain a fundamental limitation: cache management is typically formulated as a hard and irreversible token-level decision. Natural language is structured through local lexical cohesion, entity continuity, and discourse relations that extend across neighbouring tokens and sentences (Grosz 1995; Hearst 1997; Passonneau 1997; Barzilay 2008; Koshorek 2018). Contextualized representations are also strongly shaped by their surrounding context (Ethayarajh 2019), and Transformer attention captures linguistic dependencies such as syntax and coreference across related tokens (Clark et al.

*Equal Contribution

2019). Consistent with this structure, explicit modelling of contiguous spans improves tasks that require question answering, coreference resolution, and relation extraction (Joshi et al. 2020). Independently selecting individual tokens can therefore fragment locally coherent evidence by retaining isolated high-scoring states while discarding neighbouring states that contribute to their interpretation.

A second limitation is that token importance changes throughout decoding. The relevance of cached states is query-dependent, and the subset required for attention can vary substantially across decoding steps (Tang et al. 2024; Ribar et al. 2024). A state that appears unimportant under the current query may become critical later when the model resolves a reference, retrieves supporting evidence, or advances along a new reasoning trajectory. Because eviction is irreversible, information removed during a period of low apparent relevance cannot be recovered when its importance re-emerges.

Quantization preserves broader historical coverage by representing cached states at reduced precision. Existing methods, however, generally do not jointly manage cache residency and numerical precision. They lack an explicit mechanism for dynamically moving historical states among full-precision execution, compact recoverable storage, and permanent removal. This leaves a central design gap:

How can the KV cache be compressed aggressively while preserving coherent historical regions whose relevance may emerge only later?

To address this gap, we propose **QEvict**, a recoverable three-tier KV-cache hierarchy over contiguous windows. **QEvict** periodically ranks historical windows using cumulative attention, retaining high-importance windows in full precision, storing intermediate windows in a compact low-bit tier, and evicting the remainder. Quantized windows participate in attention and can be promoted when their importance re-emerges, converting eviction from a one-shot decision into a dynamic process of demotion, recovery, and promotion.

Window-level routing preserves local context and reduces sensitivity to transient token-level fluctuations, while the recoverable tier expands historical coverage under a fixed memory budget. To prevent error accumulation across repeated tier transitions, **QEvict** maintains a persistent quantized backing store: each window is quantized only on its first demotion, and the resulting low-bit representation is reused for all subsequent recoveries and promotions. The implementation supports GQA (Ainslie et al. 2023) and FlashAttention-2 (Dao 2023), while selectively materializing SDPA attention at routing steps to compute cumulative attention scores.

Our design is motivated by a systematic study of decoding-time cache dynamics. We find that historical importance is highly concentrated but temporally unstable: a small subset of windows dominates attention at any given step, yet the identity of this subset changes throughout generation, and previously inactive windows can later become relevant. To characterize this behaviour, we introduce **Future Missed Mass** and **Global LIR**, which measure future attention assigned to discarded states and the reactivation of historically inactive windows, respectively. We additionally use **Selection Churn** to quantify instability between consecutive

cache assignments. Together, these observations motivate preserving uncertain historical states without allocating full-precision memory to all of them.

We evaluate **QEvict** on LongBench (Bai et al. 2024), RULER (Hsieh et al. 2024), and GSM8K (Cobbe et al. 2021) across three instruction-tuned LLMs and multiple KV-memory budgets. The evaluation covers realistic long-context understanding, controlled retrieval and reasoning, and autoregressive multi-step generation. Across these settings, **QEvict** consistently improves the quality-memory trade-off over representative eviction and quantization baselines. Our primary contributions are:

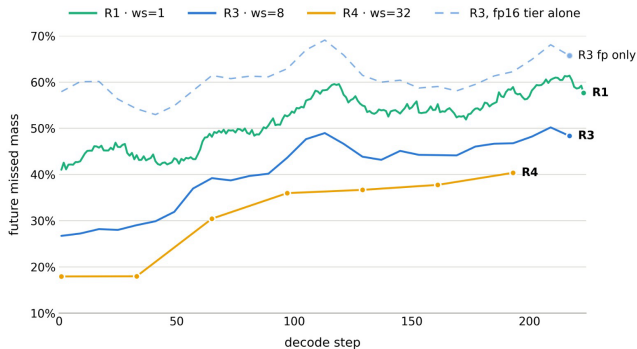
- A systematic analysis of decoding-time cache dynamics, including two new diagnostics, **Future Missed Mass** and **Global LIR**, together with **Selection Churn** for measuring routing instability.
- **QEvict**, a recoverable window-level KV-cache hierarchy that dynamically routes historical context among full-precision, low-bit, and evicted states, supported by a write-once quantization ledger.
- Extensive evaluation across three benchmarks and three model families, demonstrating improved quality-memory trade-offs under stringent KV-cache budgets.

2 Related Work

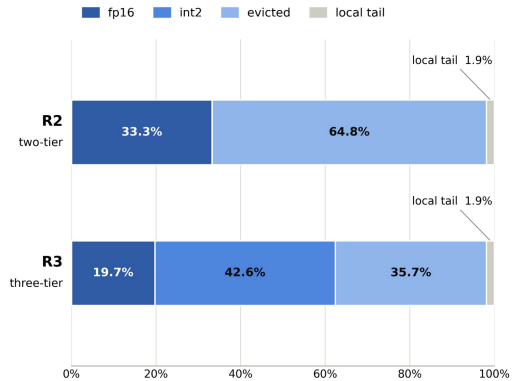
The KV cache grows linearly with sequence length and can dominate the memory cost of long-context autoregressive inference. Existing methods primarily reduce this overhead through eviction, which retains only selected historical states, or quantization, which stores a larger portion of the cache at lower precision. Eviction offers selective retention under strict budgets, whereas quantization provides broader historical coverage. **QEvict** combines these advantages through a dynamic hierarchy in which low-bit storage serves as a recoverable intermediate state.

Local Context and Dynamic Relevance. Natural language exhibits local lexical, entity, and discourse coherence across neighbouring positions (Grosz 1995; Hearst 1997; Passonneau 1997; Barzilay 2008; Koshorek 2018). Contextual representations and Transformer attention similarly encode dependencies across related spans (Ethayarajh 2019; Clark et al. 2019; Joshi et al. 2020), motivating windows as a coherent cache-management unit. Relevance is also query-dependent: **Quest** (Tang et al. 2024) and **SparQ Attention** (Ribar et al. 2024) selectively access cache regions required by the current query. **QEvict** complements these approaches by preserving historical states whose future relevance has not yet emerged.

KV-Cache Eviction. **StreamingLLM** (Xiao et al. 2024) retains initial attention sinks and a recent-token window, while **H₂O** (Zhang et al. 2023) preserves heavy-hitter tokens using accumulated attention. **SnapKV** (Li et al. 2024) estimates prompt importance from an observation window, **AdaKV** (Feng et al. 2024) adapts cache allocation across attention heads, and **CriticalKV** (Feng et al. 2025) incorporates value-related information into cache selection. **DefensiveKV** and **Layer-DefensiveKV** (Feng et al. 2026) further account for



(a) **FMM over decoding.** Window policies reduce attention assigned to discarded states. The dashed curve treats the R3 tier as inaccessible, isolating the contribution of recoverable low-bit windows.



(b) **Attention-mass allocation at a matched cache budget.** The three-tier policy preserves a broad intermediate region in INT2 and reduces the mass assigned to permanent eviction.

Figure 1: Diagnostics motivating window-level routing and recoverable low-bit retention. (a) Contiguous-window routing reduces Future Missed Mass, while the gap between the solid and dashed R3 curves shows the future utility of the quantized tier. (b) At the same KV-cache budget, three-tier allocation preserves substantially more ground-truth attention mass than binary full-precision retention and eviction.

uncertainty in future attention. Although these methods improve scoring and budget allocation, they retain a binary cache state. A token is either preserved or permanently removed. Consequently, a state discarded during a period of low apparent importance cannot be recovered when its relevance re-emerges. Token-level selection can also fragment related context by assigning different retention decisions to neighbouring states.

KV-Cache Quantization. KIVI (Liu et al. 2024) applies asymmetric 2-bit quantization with different granularities for keys and values. KVQuant (Hooper et al. 2024) combines pre-RoPE key quantization, non-uniform representations, and outlier handling for sub-4-bit compression. Zip-Cache (He et al. 2024) assigns saliency-dependent precision at token granularity.

Prior eviction and quantization methods leave two coupled challenges unresolved: token-level decisions can fragment context and fluctuate across routing steps, while static cache states cannot recover information whose importance changes during decoding. QEvict addresses these limitations by routing contiguous windows among full-precision, recoverable low-bit, and evicted states. This design preserves broader historical coverage under a fixed memory budget, stabilizes cache selection through window-level aggregation, and supports promotion and demotion as window importance evolves. Section 3 empirically examines the attention concentration, selection instability, and importance revival that motivate this three-tier hierarchy.

3 Observations and Motivation for QEvict

Conventional KV-cache eviction assumes that token-level importance is sufficiently stable for permanent deletion and that a binary retain-or-evict decision adequately captures the utility of historical states. We examine these assumptions using attention traces from an uncompressed FullKV reference,

which exposes the future attention assigned to states already removed by a compressed policy.

Diagnostic setup. We study Llama-3.1-8B-Instruct with a 512-token prefill, 256 generated tokens, a 20% KV-cache budget, and five protected sink tokens. R1 denotes token-level eviction with window size $\Omega = 1$, R2 denotes two-tier window eviction with $\Omega = 8$, and R3 and R4 denote three-tier policies with $\Omega = 8$ and $\Omega = 32$, respectively. R2 and R3 use the same window size and measured cache budget, providing a controlled comparison between binary eviction and recoverable low-bit retention. Complete experimental and metric definitions are provided in Appendix I.

Observation I: Windows stabilize cache decisions. We introduce Future Missed Mass (FMM), which measures the fraction of future FullKV attention assigned to states already discarded by a policy, and use *Selection Churn*, the Jaccard distance between historical sets retained at consecutive routing events. As shown in Figure 1a, the window-based policies incur substantially lower FMM than token-level eviction throughout decoding. Historical-set churn per routing event decreases from 0.017 for R1 to 0.0012 for R3 and 0.018 for a larger window; R4. Window aggregation therefore suppresses short-lived selection changes, although larger windows introduce a coarser allocation trade-off.

Observation II: Binary eviction discards a useful middle region. Figure 1b compares R2 and R3 at the same window size and byte budget. R2 retains 33.3% of the FullKV attention mass in full precision and evicts 64.8%. R3 retains 19.7% in full precision and preserves another 42.6% in INT2, reducing the evicted share to 35.7%. The INT2 scores maintain a cosine agreement of 0.9824 with the corresponding FullKV scores. Moreover, disabling access to the R3 quantized tier produces the FMM curve in Figure 1a, showing that these windows remain relevant to subsequent decoding.

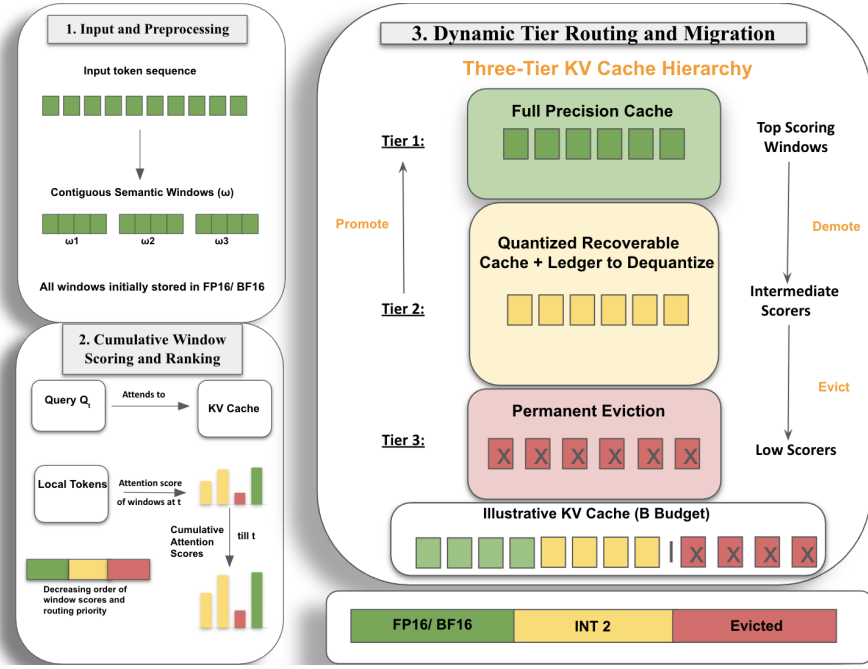


Figure 2: **Overall workflow of the QEvict framework.** Stage 1 partitions the input sequence into contiguous KV windows while preserving sink and recent tokens in full precision. Stage 2 accumulates attention scores over each window to produce a stable importance ranking. Stage 3 routes windows under a fixed byte budget into a full-precision tier, a recoverable INT2 tier that remains available for attention and later promotion, or permanent eviction, with dynamic promotion and demotion as importance evolves.

The ranked cache therefore admits three operational actions: retain in full precision, preserve compactly, or evict.

Observation III: Importance is persistent but not static. We further introduce Global LIR, which measures how often a window re-enters the full-precision set after sustained inactivity. For R3, oracle top-ranked windows exhibit a 0.98% revival rate, confirming that the dominant region is largely persistent. The deployed full-precision policy exhibits a higher 6.18% revival rate, while lagged transitions show that demotion is substantially more frequent than promotion. Thus, most important windows remain stable, but a non-negligible subset leaves and later re-enters the active set.

Together, these motivate three design choices: contiguous windows as the routing unit, a recoverable low-bit tier between full-precision retention and eviction, and bidirectional migration as importance evolves. Section 4 formalizes the resulting byte-constrained cache hierarchy.

4 Methodology

QEvict manages historical KV states as contiguous windows and dynamically assigns each window to full-precision, recoverable low-bit, or evicted state under a fixed budget.

4.1 Preliminaries and Problem Formulation

Consider transformer layer ℓ with H_q query heads and H_{kv} KV heads. At decoding step t , query $q_t^{\ell,h}$ attends over the

currently accessible keys and values:

$$a_t^{\ell,h} = \text{softmax} \left(\frac{q_t^{\ell,h} (K_{\text{acc},t}^{\ell})^\top}{\sqrt{d_{kv}}} \right), \quad o_t^{\ell,h} = a_t^{\ell,h} V_{\text{acc},t}^{\ell}.$$

After reserving a protected sink prefix and recent region, we partition the remaining historical cache into contiguous windows $\mathcal{W}_t$. For each window w , QEvict assigns a state

$$z_t^\ell(w) \in \{\text{FULL}, \text{QUANTIZED}, \text{EVICTED}\}.$$

Let $M_f(w)$ and $M_q(w)$ denote the storage costs of a full-precision and quantized window. Assignment must satisfy

$$B_{\text{sink}} + B_{\text{local}} + \sum_{w \in \mathcal{W}_t} \left[\mathbf{1}[z_t^\ell(w) = \text{FULL}] M_f(w) + \mathbf{1}[z_t^\ell(w) = \text{QUANTIZED}] M_q(w) \right] \leq B_{\text{total}}.$$

Thus, QEvict jointly determines cache residency and representation precision rather than selecting a single retained subset.

4.2 Cumulative Window Scoring

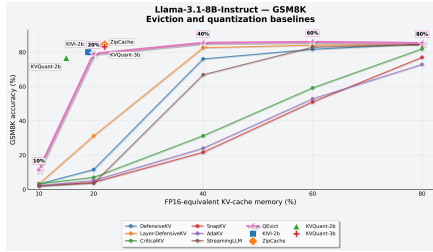
Let $a_\tau^{\ell,h}(i)$ denote the attention probability assigned to token i by query head h . At routing event t , the cumulative score of window w is updated as

$$\bar{S}_t^\ell(w) = \bar{S}_{t-\Omega}^\ell(w) + \frac{1}{H_q} \sum_{h=1}^{H_q} \sum_{\tau=t-\Omega+1}^t \sum_{i \in w} a_\tau^{\ell,h}(i), \quad (1)$$

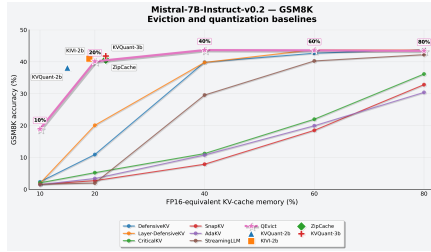
$$t \equiv 0 \pmod{R}.$$

Table 2: **RULER performance at 32K context length on Llama-3.1-8B-Instruct.** We compare QEvict with representative KV-cache eviction methods under a matched 20% cache budget and with quantization baselines at comparable memory footprints. Scores are string-match percentages, and KV memory is reported relative to the full-precision cache. Excluding Full-KV, the best and second-best distinct results within each comparison group are shown in bold and underlined, respectively.

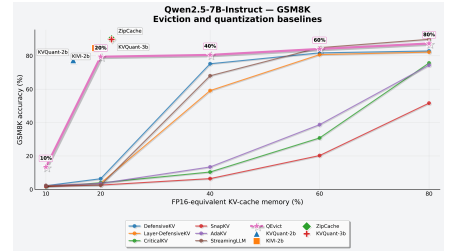
Method	KV memory	Aggregation		Needle-in-a-Haystack								Question Answering		Tracking
		CWE	FWE	MK-1	MK-2	MK-3	MQ	MV	S-1	S-2	S-3	QA-1	QA-2	VT
Llama-3.1-8B-Instruct, RULER 32K														
Full-KV	100%	45.22	94.13	99.60	99.60	99.40	98.75	99.10	100.00	100.00	100.00	79.80	54.80	99.24
EVICTION: 20% KV-CACHE BUDGET														
StreamingLLM	20%	0.04	93.40	23.20	19.20	23.00	20.90	20.10	22.20	17.60	20.80	23.40	39.00	30.76
SnapKV	20%	14.56	70.40	98.40	94.40	72.80	98.80	98.00	100.00	97.60	48.00	82.40	52.00	97.92
AdaKV	20%	19.20	75.20	99.40	93.60	88.00	99.40	99.40	100.00	99.20	56.00	77.60	51.20	99.52
CriticalKV	20%	26.80	88.80	91.60	29.40	19.40	95.00	93.60	100.00	99.60	42.40	40.80	40.20	97.76
DefensiveKV	20%	22.94	90.00	99.80	86.80	97.00	98.65	97.90	100.00	100.00	97.40	68.80	45.80	98.76
Layer-Def. KV	20%	17.86	90.80	99.60	99.40	99.00	98.85	98.45	100.00	100.00	100.00	73.00	47.60	98.56
QEvict	20%	43.98	93.60	99.80	97.00	77.80	97.45	95.90	100.00	100.00	98.80	84.60	51.20	98.84
QUANTIZATION														
QEvict	20%	43.98	93.60	99.80	97.00	77.80	97.45	95.90	100.00	100.00	98.80	84.60	51.20	98.84
KIVI-2b	~20%	20.76	91.20	98.00	90.20	21.20	92.40	96.35	99.80	98.40	90.40	75.60	50.20	81.68
ZipCache-4b (70%)	~22%	44.80	93.53	99.80	84.20	6.60	97.60	95.25	100.00	100.00	80.00	76.80	50.40	97.60
KVQuant-2b (s1%)	~15%	64.27	68.21	61.05	52.55	80.88	76.06	75.30	61.82	62.98	89.99	98.00	95.00	65.86
KVQuant-3b (s1%)	~22%	64.38	70.49	63.51	63.15	89.83	75.56	75.46	63.64	63.64	90.00	97.80	96.20	70.07



(a) Llama-3.1-8B-Instruct.



(b) Mistral-7B-Instruct-v0.2.



(c) Qwen2.5-7B-Instruct.

Figure 3: **GSM8K accuracy under KV-cache compression.** Accuracy–memory trade-offs of QEvict and representative eviction and quantization baselines across three instruction-tuned language models.

The resulting scores induce a layer-wise ranking over historical windows. Although we use cumulative attention as the default importance estimator, the hierarchy requires only an ordering and is compatible with alternative ranking functions. More detailed discussion in Appendix J

QEvict operates during both prefill and decoding. During prefill, prompt attention initializes the window scores and the first tier assignment. During decoding, newly generated states first enter the protected recent region, and historical windows are rescored and rerouted every Ω generated tokens.

4.3 QEvict: Recoverable Cache Hierarchy

At routing event t , the cache at layer ℓ is partitioned as:

$$\mathcal{C}_t^\ell = (\mathcal{S}, \mathcal{L}_t, \mathcal{F}_t, \mathcal{Q}_t, \mathcal{E}_t), \quad (2)$$

where $\mathcal{S}$ and $\mathcal{L}_t$ are the protected sink and recent regions and remain in full precision. Historical windows are assigned to the full-precision tier $\mathcal{F}_t$, the recoverable low-bit tier $\mathcal{Q}_t$, or the evicted set $\mathcal{E}_t$.

Byte-Constrained Tier Allocation. After reserving the protected regions, the remaining historical budget B_{hist} is divided between the full-precision and quantized tiers. Given quantized-tier fraction q , their capacities are

$$K_f = \left\lfloor \frac{(1-q)B_{\text{hist}}}{M_f} \right\rfloor, \quad K_q = \left\lfloor \frac{qB_{\text{hist}}}{M_q} \right\rfloor. \quad (3)$$

The per-window costs include packed codes, quantization parameters, positions, and persistent indexing state. Complete byte-level accounting and the default configuration are provided in Appendix B.4 and Appendix B.5. Sensitivity to q , Ω , and quantization precision is studied in Appendix G.

Migration-Stable Quantization. When a window first enters $\mathcal{Q}_t$, QEvict applies asymmetric low-bit quantization. Following the distinct distributions of keys and values (Liu et al. 2024), keys are quantized per channel across the token dimension, whereas values are quantized per token across

Table 3: **LongBench performance on Llama-3.1-8B-Instruct and Mistral-7B-Instruct-v0.2.** Each cell reports the Llama score followed by the Mistral score. We compare Q_{Evict} with representative KV-cache eviction methods under matched 20%, 10%, and 5% budgets, and with quantization baselines at comparable memory footprints. KV memory is reported relative to the full-precision cache. Excluding Full-KV, the best and second-best distinct results for each model within every comparison group are shown in bold and underlined, respectively.

Method	KV memory	Single-Doc QA			Multi-Doc QA			Summarization			Few-Shot Learning		
		Nar.QA	Qasper	Mul.QA	Hot.QA	2Wi.QA	Musique	Gov.Re.	QMSum	M.News	TREC	Tri.QA	SAMSum
Llama-3.1-8B-Instruct / Mistral-7B-Instruct-v0.2													
Full-KV	100%	30.5/21.0	45.5/29.4	55.0/47.1	56.0/36.5	45.7/21.8	31.3/19.1	35.1/32.6	25.6/24.0	27.3/27.1	73.0/71.0	91.7/86.2	43.7/43.0
EVICTION: 20% KV-CACHE BUDGET													
StreamingLLM	20%	19.9/15.3	19.6/13.3	24.4/23.9	40.5/27.8	20.3/14.7	15.1/11.6	27.8/27.4	20.3/20.1	22.3/21.9	53.5/43.0	89.7/76.9	40.0/40.2
SnapKV	20%	20.9/16.7	28.3/17.1	29.4/31.1	45.7/30.1	30.0/16.8	22.8/11.5	27.3/26.2	21.6/21.4	22.6/22.8	50.0/51.0	92.2/87.2	44.6/42.3
AdaKV	20%	21.7/17.1	28.4/17.6	33.5/32.3	50.8/30.8	30.4/17.2	21.7/13.4	26.4/25.8	21.6/21.1	22.9/22.9	55.0/53.0	91.4/87.0	43.5/43.2
CriticalKV	20%	29.9/19.6	31.1/19.5	33.2/33.8	51.3/30.7	34.3/18.2	24.9/13.4	28.5/27.6	22.9/21.7	23.1/23.3	55.0/57.5	91.8/87.2	44.1/41.8
DefensiveKV	20%	27.1/19.2	40.8/23.1	47.8/43.7	55.3/34.3	38.7/21.7	27.9/17.3	29.2/28.6	23.7/21.7	23.9/23.7	66.0/67.0	91.8/86.7	43.7/43.8
Layer-Def. KV	20%	27.7/20.5	44.3/26.0	50.7/47.0	53.2/35.3	40.7/23.3	27.5/18.0	30.6/30.3	24.1/22.7	24.6/24.7	69.0/71.0	91.6/86.6	43.7/43.5
QEvict	20%	29.69/22.6	42.3/27.7	55.74/46.9	57.97/38.0	48.05/20.8	32.99/18.3	33.71/30.8	25.11/24.1	26.89/26.8	71.0/71.0	91.21/85.5	41.7/41.1
EVICTION: 10% KV-CACHE BUDGET													
StreamingLLM	10%	18.2/15.8	16.6/10.8	22.8/22.6	34.7/23.1	16.6/14.3	11.6/10.3	24.7/24.8	18.8/19.2	19.8/18.8	51.0/32.0	88.8/72.8	38.3/38.6
SnapKV	10%	18.9/14.9	20.8/12.0	23.6/25.2	43.0/26.4	22.6/13.5	18.2/10.8	24.1/24.3	19.6/20.2	20.4/20.9	44.5/44.5	92.9/87.3	43.0/40.7
AdaKV	10%	18.3/16.3	22.7/13.7	26.1/27.2	40.8/29.1	22.2/15.4	17.8/11.9	24.3/24.3	20.2/19.9	20.9/20.6	46.5/48.5	92.2/87.1	43.3/42.2
CriticalKV	10%	25.1/16.5	23.4/13.9	26.1/26.4	40.3/27.6	25.6/15.6	19.7/10.5	25.3/25.4	21.0/20.6	20.9/21.5	45.5/45.5	92.5/87.6	42.5/42.0
DefensiveKV	10%	20.9/17.0	26.0/14.8	34.0/32.6	48.3/31.7	30.0/16.7	19.6/11.6	25.1/25.1	22.4/20.8	21.5/21.6	54.0/53.0	91.4/87.4	43.7/43.1
Layer-Def. KV	10%	22.6/19.2	29.1/16.7	37.5/38.6	49.6/35.6	30.2/20.6	21.7/14.7	25.6/26.8	22.6/21.4	21.4/21.9	52.0/64.0	91.5/87.4	44.0/43.4
QEvict	10%	28.8/20.1	34.94/23.3	49.8/42.6	58.4/36.5	46.4/18.7	32.9/18.0	32.1/29.6	24.9/24.0	24.6/25.1	67.5/69.0	89.8/83.4	40.4/39.5
EVICTION: 5% KV-CACHE BUDGET													
StreamingLLM	5%	14.3/13.1	13.9/9.4	19.7/20.3	30.1/18.5	14.7/12.8	7.9/8.2	22.2/21.8	17.6/18.8	17.3/15.9	38.5/19.5	87.8/68.7	36.2/37.5
SnapKV	5%	18.3/14.4	15.7/11.4	19.8/21.8	36.3/24.0	13.4/14.7	13.2/8.6	21.6/21.8	18.3/19.2	17.8/17.8	34.0/38.5	92.0/86.6	42.1/39.5
AdaKV	5%	17.0/15.1	16.4/10.6	19.6/22.9	36.7/24.8	17.1/14.5	14.4/9.0	21.7/21.6	18.7/19.4	18.4/18.2	35.5/41.0	92.8/87.4	42.0/39.8
CriticalKV	5%	20.2/14.7	17.4/11.0	19.6/22.9	36.0/25.8	15.5/14.3	15.8/9.2	22.7/23.0	18.7/19.4	18.2/18.2	34.0/40.0	92.5/85.7	42.0/40.9
DefensiveKV	5%	20.8/16.1	18.1/11.1	21.6/24.3	39.8/25.7	19.8/15.4	14.2/10.5	23.2/23.1	19.5/20.0	18.9/19.0	43.5/44.0	93.0/87.1	43.2/40.9
Layer-Def. KV	5%	19.8/15.6	20.1/11.8	21.8/27.0	40.9/28.0	19.8/14.9	17.8/10.3	22.9/23.9	19.7/20.4	18.9/19.2	41.5/47.0	93.0/87.6	43.2/42.6
QEvict	5%	27.3/20.0	28.0/16.8	43.5/34.6	56.4/34.1	45.1/17.1	32.5/25.1	28.8/27.6	23.9/22.7	19.3/19.9	63.0/63.5	89.6/84.6	38.2/38.2
QUANTIZATION													
QEvict	10%	28.8/20.1	34.94/23.3	49.8/42.6	58.4/36.5	46.4/18.7	32.9/18.0	32.1/29.6	24.9/24.0	24.6/25.1	67.5/69.0	89.8/83.4	40.4/39.5
QEvict	20%	29.69/22.6	42.3/27.7	55.74/46.9	57.97/38.0	48.05/20.8	32.99/18.3	33.71/30.8	25.11/24.1	26.89/26.8	71.0/71.0	91.21/85.5	41.7/41.1
KIVI-2b	~20%	23.7/20.6	36.8/28.7	41.1/44.9	44.4/35.5	30.0/20.7	21.9/18.0	30.5/32.6	24.4/23.7	26.0/26.5	68.0/71.0	87.8/86.0	44.7/43.3
ZipCache-4b (70%)	~22%	24.1/20.4	43.3/28.1	53.9/45.4	47.4/33.2	50.8/20.9	25.2/16.6	32.6/29.9	22.9/22.9	26.5/26.8	21.0/39.5	91.0/80.3	20.4/37.3
KVQuant-2b (s1%)	~15%	22.6/16.0	36.5/23.4	47.1/41.8	43.8/29.3	34.2/17.3	23.2/11.6	31.5/27.5	22.9/22.6	25.8/26.0	65.0/10.8	86.3/68.0	41.9/25.1
KVQuant-3b (s1%)	~22%	28.6/20.1	44.3/28.6	53.4/47.1	53.9/36.7	42.7/19.9	28.0/17.3	33.4/30.4	25.0/23.7	26.5/26.7	72.5/10.3	92.0/77.1	44.6/37.0

the channel dimension. Keys are represented in the pre-ROPE domain together with their original absolute positions, and RoPE is reapplied after dequantization.

The codes and quantization parameters produced on the first demotion are retained and reused during later tier transitions. Promotion reconstructs the same low-bit approximation in the model’s execution datatype, while a subsequent demotion reuses the existing representation instead of re-quantizing the reconstructed values. This prevents approximation error from compounding across repeated migrations.

Dynamic Routing and Recovery. At each routing event, the candidate pool contains the current full-precision and quantized windows together with windows that have exited the recent region:

$$\mathcal{W}_t^{\text{cand}} = \mathcal{F}_{t-\Omega} \cup \mathcal{Q}_{t-\Omega} \cup \mathcal{L}_t^{\text{aged}}. \quad (4)$$

Candidates are ranked by $\bar{S}_t^\ell(w)$. Highest-ranked K_f windows are assigned to $\mathcal{F}_t$, next K_q to $\mathcal{Q}_t$, and rest to $\mathcal{E}_t$:

$$\mathcal{F}_t = \text{TopK}_{K_f}(\mathcal{W}_t^{\text{cand}}; \bar{S}_t^\ell)$$

$$\begin{aligned} \mathcal{Q}_t &= \text{TopK}_{K_q}(\mathcal{W}_t^{\text{cand}} \setminus \mathcal{F}_t; \bar{S}_t^\ell), \\ \mathcal{E}_t &= \mathcal{W}_t^{\text{cand}} \setminus (\mathcal{F}_t \cup \mathcal{Q}_t). \end{aligned} \quad (5)$$

Quantized windows remain available to attention and continue accumulating scores. A window can therefore be promoted when its importance increases or demoted when it declines:

$$\text{FULL} \rightleftharpoons \text{QUANTIZED} \longrightarrow \text{EVICTED}. \quad (6)$$

Only windows outside the combined capacities $K_f + K_q$ are permanently removed.

Cache Execution. Full-precision and quantized windows are stored separately. Before attention, active low-bit windows are dequantized, their keys are re-rotated at the original positions, and all accessible states are restored in chronological order. Standard decoding uses FlashAttention-2, while SDPA is invoked only at prefill initialization and routing events to expose the attention probabilities required for scoring. Further execution details are provided in Appendix H.

5 Experiments

Setup. We evaluate `QEvict` on Llama-3.1-8B-Instruct (Dubey et al. 2024), Qwen2.5-7B-Instruct (Yang et al. 2025), and Mistral-7B-Instruct-v0.2 (Jiang et al. 2023) using LongBench (Bai et al. 2024), RULER (Hsieh et al. 2024), and GSM8K (Cobbe et al. 2021). We compare against two baseline families: permanent KV-cache eviction methods and global KV-cache quantization methods. Eviction baselines are evaluated under matched measured-memory budgets, while quantization baselines are compared at their corresponding performance–memory operating points.

Unless otherwise stated, `QEvict` uses routing interval $\Omega = 8$, five sink tokens, 32 recent tokens, an INT2 recoverable tier, and quantized-tier fraction $q = 0.70$ across LongBench, RULER, and GSM8K. Complete baseline configurations, decoding protocols, and byte-level memory accounting are provided in Appendix B.

Long-context understanding. We evaluate 12 LongBench tasks across single-document QA, multi-document QA, summarization, and few-shot learning at 5%, 10%, and 20% KV-memory budgets. Table 3 reports results for Llama and Mistral, with Qwen results in Appendix C.

`QEvict` achieves the highest macro-average across all six model–budget settings. At 20% memory, it scores 46.4 on Llama and 37.8 on Mistral, compared with 44.0 and 37.4 for the strongest matched-memory eviction baseline. The gains increase at 5% memory to 9.7 and 4.7 points, respectively, highlighting the value of recoverable low-bit retention under tight budgets. `QEvict` also achieves the best 20% macro-average among quantization methods; at only 10% memory, it remains within 1.2 points on Llama and 1.8 points on Mistral of the strongest quantization baseline despite using roughly half the KV memory. Further analyses are provided in Appendix D.

Long-range retrieval. We evaluate RULER at a 32K context length and a 20% KV-cache budget on Llama-3.1-8B-Instruct. Table 2 reports all 13 aggregation, needle-in-a-haystack, question-answering, and tracking tasks. `QEvict` obtains a macro-average of 87.6, exceeding the strongest matched-memory eviction baseline, Layer-DefensiveKV, by 1.2 points. It also outperforms the strongest comparable-memory quantization baseline by 8.6 points and remains within 2.4 points of the uncompressed Full-KV reference. The results show that broad low-bit historical coverage is especially effective for retrieval-intensive long-context tasks.

Reasoning under compression. On GSM8K, Figure 3 reports accuracy–memory curves for Llama, Qwen, and Mistral over complete memory sweep. This evaluates whether the recoverable hierarchy preserves multi-step generation quality as the available cache budget decreases. Complete numerical results and model-specific comparisons with eviction and quantization baselines are provided in Appendix F.

Ablations. We vary the routing interval Ω , quantized-tier fraction q , and recoverable precision. The results favour $\Omega = 8$ and $q = 0.70$ across benchmarks. INT2 provides broader historical coverage under a fixed byte budget, while INT4

Table 4: **End-to-end inference efficiency on Llama-3.1-8B-Instruct.** Results use a 256-token prefill, 1024 generated tokens, batch size 32, and the default `QEvict` configuration. Comparisons are made within each attention backend.

Backend	Method	TTFT ↓ (ms)	TPOT ↓ (ms)	Throughput ↑ (tok/s)	Peak GPU ↓ (GB)
FlashAttention-2	Full-KV	675.0	59.74	535.11	29.54
	QEvict	772.0	146.35	218.05	20.78
Eager/SDPA	Full-KV	689.3	84.05	379.26	24.38
	QEvict	692.4	76.26	416.44	24.36

trades coverage for lower quantization error. Full results are reported in Appendix G.

5.1 Efficiency, Limitations, and Future Work

End-to-end efficiency. Table 4 evaluates `QEvict` under matched model, batch, sequence-length, and backend settings. With eager SDPA, `QEvict` adds only 0.5% TTFT overhead, reduces TPOT by 9.3%, and improves decoding throughput by 9.8% over Full-KV. Thus, reducing the full-precision attended cache offsets the cost of low-bit cache management in the eager implementation.

With FlashAttention-2, `QEvict` reduces peak GPU memory from 29.54 to 20.78 GB, a 29.7% reduction. Its current execution path, however, requires periodic attention-score materialization together with dequantization and cache reconstruction, which lowers decoding throughput. The resulting trade-off is backend dependent: `QEvict` improves eager decoding efficiency and substantially reduces FlashAttention-2 memory, while fused low-bit attention and routing kernels remain necessary to realize both benefits simultaneously.

Limitations and future work. The current implementation reconstructs active low-bit windows before attention and selectively falls back to SDPA at routing events to expose attention probabilities. These operations limit FlashAttention-2 throughput and introduce temporary workspace overhead. Future work will integrate dequantization, positional rotation, and mixed-precision attention directly into fused kernels, and investigate lower-cost importance estimators that avoid explicit attention-score materialization. The present evaluation also focuses on decoder-only models and fixed-size contiguous windows, leaving adaptive window boundaries and broader model architectures for future study.

6 Conclusion

We introduced `QEvict`, a byte-constrained KV-cache framework that routes historical windows across full-precision, recoverable INT2, and evicted tiers. Across LongBench, RULER, and GSM8K, `QEvict` improves performance–memory trade-offs over eviction and quantization baselines, while reducing peak GPU memory and improving eager decoding throughput. These results show that recoverable low-bit retention is a practical alternative to permanent eviction.

References

- Ainslie, J.; Lee-Thorp, J.; De Jong, M.; Zemlyanskiy, Y.; Lebrón, F.; and Sanghai, S. 2023. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 4895–4901.
- Bai, Y.; Lv, X.; Zhang, J.; Lyu, H.; Tang, J.; Huang, Z.; Du, Z.; Liu, X.; Zeng, A.; Hou, L.; Dong, Y.; Tang, J.; and Li, J. 2024. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.
- Barzilay. 2008. Modeling Local Coherence: An Entity-Based Approach. *Computational Linguistics*, 34(1): 1–34.
- Brown, T. B.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, 1877–1901.
- Clark, K.; Khandelwal, U.; Levy, O.; and Manning, C. D. 2019. What Does BERT Look at? An Analysis of BERT’s Attention. In Linzen, T.; Chrupała, G.; Belinkov, Y.; and Hupkes, D., eds., *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, 276–286. Florence, Italy: Association for Computational Linguistics.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Dao, T. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691*.
- Devoto, A.; Zhao, Y.; Scardapane, S.; and Minervini, P. 2024. A simple and effective l_2 norm-based strategy for kv cache compression. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 18476–18499.
- Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*.
- Ethayarajh, K. 2019. How Contextual are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings. In Inui, K.; Jiang, J.; Ng, V.; and Wan, X., eds., *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 55–65. Hong Kong, China: Association for Computational Linguistics.
- Feng, Y.; Guo, H.; Lv, J.; Zhou, S. K.; and Xie, X. 2026. DefensiveKV: Taming the fragility of KV cache eviction in LLM inference. In *The Fourteenth International Conference on Learning Representations, 2026a*. URL <https://openreview.net/forum>.
- Feng, Y.; Lv, J.; Cao, Y.; Xie, X.; and Zhou, S. K. 2024. Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference.
- Feng, Y.; Lv, J.; Guo, H.; Cao, Y.; Xie, X.; and Zhou, S. K. 2025. Identify Critical KV Cache in LLM Inference from an Output Perturbation Perspective. *OpenReview preprint*.
- Grosz. 1995. Centering: A Framework for Modeling the Local Coherence of Discourse. *Computational Linguistics*, 21(2): 203–225.
- He, Y.; Zhang, L.; Wu, W.; Liu, J.; Zhou, H.; and Zhuang, B. 2024. Zipcache: Accurate and efficient kv cache quantization with salient token identification. *Advances in Neural Information Processing Systems*, 37: 68287–68307.
- Hearst, M. A. 1997. Text Tiling: Segmenting Text into Multi-paragraph Subtopic Passages. *Computational Linguistics*, 23(1): 33–64.
- Hooper, C.; Kim, S.; Mohammadzadeh, H.; Mahoney, M. W.; Shao, S.; Keutzer, K.; and Gholami, A. 2024. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. In *NeurIPS*.
- Hsieh, C.-P.; Sun, S.; Krizan, S.; Acharya, S.; Rekesh, D.; Jia, F.; Zhang, Y.; and Ginsburg, B. 2024. RULER: What’s the Real Context Size of Your Long-Context Language Models? In *Conference on Language Modeling*.
- Jiang, A. Q.; Sablayrolles, A.; Mensch, A.; Bamford, C.; Chaplot, D. S.; de las Casas, D.; Bressand, F.; Lengyel, G.; Lample, G.; Saulnier, L.; et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825*.
- Joshi, M.; Chen, D.; Liu, Y.; Weld, D. S.; Zettlemoyer, L.; and Levy, O. 2020. SpanBERT: Improving Pre-training by Representing and Predicting Spans. *Transactions of the Association for Computational Linguistics*, 8: 64–77.
- Koshorek. 2018. Text Segmentation as a Supervised Learning Task. In Walker, M.; Ji, H.; and Stent, A., eds., *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, 469–473. New Orleans, Louisiana: Association for Computational Linguistics.
- Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C. H.; Gonzalez, J.; Zhang, H.; and Stoica, I. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*.
- Li, Y.; Huang, Y.; Yang, B.; Venkitesh, B.; Locatelli, A.; Ye, H.; Cai, T.; Lewis, P.; and Chen, D. 2024. SnapKV: LLM Knows What You are Looking for Before Generation. In *NeurIPS*.
- Liu, Z.; Desai, A.; Liao, F.; Wang, W.; Xie, V.; Xu, Z.; Kyrillidis, A.; and Shrivastava, A. 2023. Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. In *NeurIPS*.
- Liu, Z.; Yuan, J.; Jin, H.; Zhong, S.; Xu, Z.; Braverman, V.; Chen, B.; and Hu, X. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache.
- Mirzadeh, I.; Alizadeh, K.; Shahrokhi, H.; Tuzel, O.; Bengio, S.; and Farajtabar, M. 2024. Gsm-symbolic: Understanding the limitations of mathematical reasoning in large language models. *arXiv preprint arXiv:2410.05229*.

Passonneau. 1997. Discourse Segmentation by Human and Automated Means. *Computational Linguistics*, 23(1): 103–139.

Pope, R.; Douglas, S.; Chowdhery, A.; Devlin, J.; Bradbury, J.; Heek, J.; Xiao, K.; Agrawal, S.; and Dean, J. 2023. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5: 606–624.

Ribar, L.; Chelombiev, I.; Hudlass-Galley, L.; Blake, C.; Luschi, C.; and Orr, D. 2024. SparQ Attention: Bandwidth-Efficient LLM Inference. In Salakhutdinov, R.; Kolter, Z.; Heller, K.; Weller, A.; Oliver, N.; Scarlett, J.; and Berkenkamp, F., eds., *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 42558–42583. PMLR.

Tang, J.; Zhao, Y.; Zhu, K.; Xiao, G.; Kasikci, B.; and Han, S. 2024. QUEST: Query-Aware Sparsity for Efficient Long-Context LLM Inference. In Salakhutdinov, R.; Kolter, Z.; Heller, K.; Weller, A.; Oliver, N.; Scarlett, J.; and Berkenkamp, F., eds., *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 47901–47911. PMLR.

Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.-A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, L.; and Polosukhin, I. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, volume 30.

Xiao, G.; Tian, Y.; Chen, B.; Han, S.; and Lewis, M. 2024. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations*.

Yang, A.; Li, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Gao, C.; Huang, C.; Lv, C.; et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Zhang, Z.; Sheng, Y.; Zhou, T.; Chen, T.; Zheng, L.; Cai, R.; Song, Z.; Tian, Y.; Ré, C.; Barrett, C.; et al. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *NeurIPS*.

A Algorithm

Algorithm 1 QEvict Dynamic Window Routing

Require: Previous tiers $\mathcal{F}_{t-\Omega}$ and $\mathcal{Q}_{t-\Omega}$

Require: Aged local windows $\mathcal{L}_{\text{aged}}$, scores $\bar{S}_t(w)$, capacities K_f, K_q , ledger Λ

```

1:  $\mathcal{C}_{\text{cand}} \leftarrow \mathcal{F}_{t-\Omega} \cup \mathcal{Q}_{t-\Omega} \cup \mathcal{L}_{\text{aged}}$ 
2:  $\mathcal{F}_t \leftarrow \text{TopK}_{K_f} \{\bar{S}_t(w) : w \in \mathcal{C}_{\text{cand}}\}$ 
3:  $\mathcal{Q}_t \leftarrow \text{TopK}_{K_q} \{\bar{S}_t(w) : w \in \mathcal{C}_{\text{cand}} \setminus \mathcal{F}_t\}$ 
4:  $\mathcal{E}_t \leftarrow \mathcal{C}_{\text{cand}} \setminus (\mathcal{F}_t \cup \mathcal{Q}_t)$ 
5: for  $w \in \mathcal{F}_t$  do
6:   if  $w \in \mathcal{Q}_{t-\Omega}$  then
7:      $(\hat{K}_w, \hat{V}_w) \leftarrow \text{Dequantize}(\Lambda_w)$ 
8:     mark  $\Lambda_w$  dormant
9:   end if
10: end for
11: for  $w \in \mathcal{Q}_t$  do
12:   if  $w$  has no ledger entry then
13:     quantize  $w$  once and create  $\Lambda_w$ 
14:   else if  $\Lambda_w$  is dormant then
15:     reactivate  $\Lambda_w$ 
16:   end if
17: end for
18: for  $w \in \mathcal{E}_t$  do
19:   release  $w$  and delete  $\Lambda_w$  if present
20: end for
21: reconstruct the effective cache in chronological order
22: return  $(\mathcal{F}_t, \mathcal{Q}_t, \mathcal{E}_t, \Lambda)$ 

```

B Complete Experimental Details

This section specifies the models, inference protocol, baselines, comparison regimes, memory accounting, and default QEvict configuration used throughout the evaluation.

B.1 Models and Inference Protocol

We evaluate QEvict on Llama-3.1-8B-Instruct (Dubey et al. 2024), Qwen2.5-7B-Instruct (Yang et al. 2025), and Mistral-7B-Instruct-v0.2 (Jiang et al. 2023). QEvict is training-free and modifies only KV-cache management at inference time. Within each benchmark, all methods use the same checkpoint, tokenizer, prompt template, input and generation limits, and answer post-processing. Unless stated otherwise, decoding is greedy.

We evaluate three complementary capabilities: long-context understanding on LongBench (Bai et al. 2024), controlled retrieval and aggregation on RULER (Hsieh et al. 2024), and multi-step mathematical reasoning on GSM8K (Cobbe et al. 2021).

B.2 Baselines

We compare against eviction and quantization methods, whose technical details are reviewed in Section 2.

Eviction baselines. The eviction baselines are StreamingLLM (Xiao et al. 2024), SnapKV (Li et al. 2024), AdaKV (Feng et al. 2024), CriticalKV (Feng et al. 2025), DefensiveKV, and Layer-DefensiveKV (Feng et al. 2026). They cover sink- and recency-based retention, accumulated-attention scoring, prompt observation, head-adaptive allocation, value-aware importance, and drift-aware eviction.

Quantization baselines. We compare against KIVI (Liu et al. 2024), KVQuant (Hooper et al. 2024), and ZipCache (He et al. 2024). These methods retain broad historical coverage at reduced precision, whereas QEvict jointly determines residency and precision through full-precision, quantized, and evicted tiers.

Uncompressed reference. Full-KV stores the complete key and value cache in the model’s native inference datatype and serves as the uncompressed reference.

B.3 Comparison Regimes

Eviction and quantization methods are evaluated under separate protocols because nominal compression settings do not necessarily yield the same physical memory footprint.

Eviction baselines. For every method $\mathcal{A}$, we define the measured KV-memory ratio as $\rho_{\mathcal{A}} = M_{\mathcal{A}}/M_{\text{FullKV}}$. LongBench uses $\rho_{\mathcal{A}} \in \{0.05, 0.10, 0.20\}$, RULER 32K uses $\rho_{\mathcal{A}} = 0.20$, and GSM8K uses $\rho_{\mathcal{A}} \in \{0.10, 0.20, 0.40, 0.60, 0.80, 1.0\}$ for the eviction comparisons.

Quantization baselines. Quantization methods are compared at their measured performance–memory operating points rather than by nominal bit width.

B.4 Memory Accounting

For a model with L layers, H_{kv} KV heads, head dimension d_h , sequence length S , and c_f bytes per element, the uncompressed KV-cache footprint is

$$M_{\text{FullKV}} = 2LH_{kv}d_hSc_f, \quad (7)$$

where the factor of two accounts for keys and values.

All persistent cache-related storage is included in the measured budget, including retained KV tensors, packed codes, quantization parameters, full-precision residual regions, outliers, and indexing metadata when applicable. For QEvict,

$$M_{\text{QEvict}} = M_{\text{sink}} + M_{\text{recent}} + M_{\text{FP}} + M_{\text{Q}} + M_{\text{quant-meta}} + M_{\text{position}} + M_{\text{ledger}}. \quad (8)$$

Here, M_{Q} contains packed low-bit keys and values, $M_{\text{quant-meta}}$ contains their scales and offsets, and M_{ledger} contains persistent migration records. Temporary workspaces are excluded from the KV-cache budget and captured separately by peak GPU memory.

B.5 QEvict Configuration

Unless explicitly varied, QEvict uses routing interval $\Omega = 8$, five sink tokens, and an INT2 recoverable tier. After allocating the protected regions, a fraction $q = 0.70$ of the remaining historical budget is assigned to the quantized tier and $1 - q$ to the full-precision historical tier. This configuration is used for LongBench, RULER, and GSM8K. Sensitivity to Ω , q , and quantization precision is reported in Appendix G.

C LongBench Evaluation

C.1 Task Families and Metrics

We evaluate 12 LongBench tasks spanning the four families summarized in Table 5 and we store 128 recent tokens for this task.

Table 5: LongBench task families used in our evaluation.

Task Family	Datasets	Evaluation Focus
Single-Document QA	NarrativeQA, Qasper, MultiFieldQA-en	Retrieving evidence from one long document.
Multi-Document QA	HotpotQA, 2WikiMultiHopQA, MuSiQue	Retrieving and combining evidence across documents.
Summarization	GovReport, QMSum, MultiNews	Preserving information distributed throughout the input.
Few-Shot Learning	TREC, TriviaQA, SAMSum	Retaining demonstrations and task structure.

We use the official metric for each dataset and report task-level scores.

C.2 Complete Results

All baselines, configurations, memory-accounting rules, and comparison protocols follow Appendix B, particularly Appendix B.3. Results for LongBench on Llama-3.1-8B-Instruct (Dubey et al. 2024) and Mistral-7B-Instruct-v0.2 (Jiang et al. 2023) are reported in the main-paper Table 3 and for Qwen2.5-7B-Instruct (Yang et al. 2025) is reported in Table 6.

D LongBench Performance–Memory Pareto Analysis

Fixed-budget comparisons capture performance at only a small number of operating points and may obscure the overall trade-off between task quality and KV-cache memory. We therefore analyse the performance–memory Pareto frontier for each LongBench task family, model, and baseline class.

Let (ρ_i, S_i) denote an operating point, where ρ_i is the measured KV-memory ratio and S_i is the corresponding task-family score. Operating point i dominates j when

$$\rho_i \leq \rho_j, \quad S_i \geq S_j, \quad (9)$$

Table 6: **LongBench performance on Qwen2.5-7B-Instruct.** We compare Q_{Evict} with representative KV-cache eviction methods under matched 20%, 10%, and 5% budgets, and with quantization baselines at comparable memory footprints. KV memory is reported relative to the full-precision cache. Excluding Full-KV, the best and second-best distinct results within each comparison group are shown in bold and underlined, respectively.

Method	KV memory	Single-Doc QA			Multi-Doc QA			Summarization			Few-Shot Learning		
		Nar.QA	Qasper	Mul.QA	Hot.QA	2Wi.QA	Musique	Gov.Re.	QMSum	M.News	TREC	Tri.QA	SAMSum
Qwen2.5-7B-Instruct													
Full-KV	100%	29.33	46.36	50.27	55.96	42.49	27.55	33.73	24.27	25.41	73.50	86.51	41.20
EVICTION: 20% KV-CACHE BUDGET													
StreamingLLM	20%	18.56	20.05	23.83	35.02	27.09	14.34	28.50	18.99	21.05	57.00	78.49	42.96
SnapKV	20%	22.21	22.43	30.67	45.89	28.77	22.00	28.98	19.94	21.28	49.00	87.61	40.20
AdaKV	20%	23.35	22.53	29.57	44.57	28.65	21.37	28.61	20.09	21.45	53.50	87.89	40.00
CriticalKV	20%	22.82	26.72	31.74	46.85	37.09	24.95	29.75	20.73	22.15	53.00	88.94	45.61
DefensiveKV	20%	20.80	30.30	34.60	44.30	32.00	19.20	30.40	20.60	22.40	37.50	87.20	46.10
Layer-Def. KV	20%	20.46	30.45	39.34	46.54	36.16	21.28	31.03	21.49	22.50	61.00	87.23	45.36
QEvict	20%	19.97	35.74	44.28	49.15	37.43	24.37	26.68	22.04	25.28	67.00	89.63	44.17
EVICTION: 10% KV-CACHE BUDGET													
StreamingLLM	10%	18.20	16.01	21.25	27.39	22.83	11.38	25.88	18.30	17.87	49.50	71.06	41.03
SnapKV	10%	19.43	14.06	24.55	36.48	23.98	17.13	26.35	18.34	18.69	44.50	87.71	39.17
AdaKV	10%	20.78	15.22	25.67	37.27	22.13	17.00	26.27	18.09	19.04	43.50	87.14	39.67
CriticalKV	10%	19.90	19.67	25.33	40.75	27.36	19.21	27.02	18.83	19.99	46.50	88.64	45.18
DefensiveKV	10%	19.00	20.10	26.50	34.00	31.70	16.40	28.00	19.10	20.20	27.50	87.90	43.60
Layer-Def. KV	10%	21.19	21.93	26.81	37.53	22.32	17.43	20.44	18.83	21.99	43.50	87.51	44.62
QEvict	10%	23.88	34.46	37.71	49.22	37.13	23.56	29.35	22.10	23.72	64.50	83.43	42.55
EVICTION: 5% KV-CACHE BUDGET													
StreamingLLM	5%	11.98	12.02	20.28	28.05	22.13	9.87	23.42	17.10	15.82	38.50	45.69	34.11
SnapKV	5%	18.17	14.35	19.85	29.09	21.97	14.30	24.46	17.46	16.80	29.50	86.00	41.51
AdaKV	5%	17.46	14.41	21.06	29.60	22.84	12.85	24.42	17.32	17.32	33.00	87.80	41.54
CriticalKV	5%	17.87	14.48	21.77	35.36	20.39	14.37	24.39	17.38	17.03	36.00	88.45	43.11
DefensiveKV	5%	16.80	14.90	21.20	30.50	24.80	14.30	25.60	17.90	17.60	27.50	87.90	43.20
Layer-Def. KV	5%	15.86	14.33	22.12	30.51	24.50	14.94	25.24	17.19	17.08	34.00	87.90	42.05
QEvict	5%	23.73	28.61	33.78	49.77	35.16	22.89	28.74	21.85	18.54	59.50	84.93	41.48
QUANTIZATION													
QEvict	10%	23.88	34.46	37.71	49.22	37.13	23.56	29.35	22.10	23.72	64.50	83.43	42.55
QEvict	20%	19.97	35.74	44.28	49.15	37.43	24.37	26.68	22.04	25.28	67.00	89.63	44.17
KIVI-2b	~20%	24.23	28.25	42.64	45.90	38.07	24.15	25.41	22.17	22.55	65.25	76.21	43.94
ZipCache-4b (70%)	~22%	22.82	34.53	45.81	48.92	40.03	23.50	29.10	21.65	23.49	34.50	90.97	40.15
KVQuant-2b (s1%)	~15%	20.98	32.57	42.58	41.53	32.25	16.00	28.99	21.93	23.02	57.25	79.54	35.96
KVQuant-3b (s1%)	~22%	24.19	37.07	45.16	53.94	39.43	26.24	31.38	23.59	23.86	69.00	85.44	39.33

with at least one strict inequality. The Pareto frontier is the set of non-dominated operating points. Points closer to the upper-left corner are preferable because they achieve higher task performance with lower KV-memory usage.

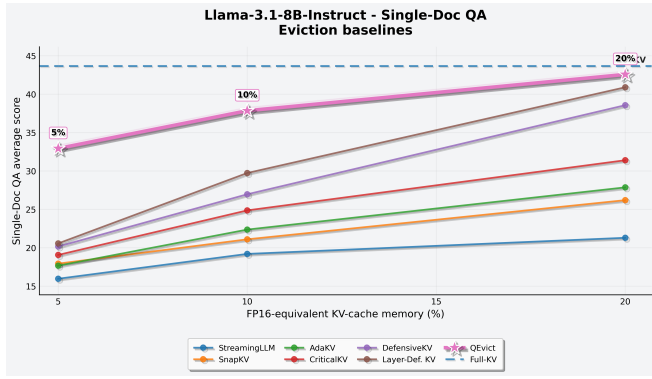
We construct separate comparisons against eviction and quantization baselines for Single-Document QA, Multi-Document QA, Summarization, and Few-Shot Learning. This produces eight panels per model and 24 panels across Llama-3.1-8B-Instruct, Mistral-7B-Instruct-v0.2, and Qwen2.5-7B-Instruct. Figures 4, 5, and 6 report the corresponding Pareto plots. The left column of each figure compares Q_{Evict} with eviction methods, while the right column compares it with quantization methods. Horizontal dashed lines indicate the performance of the uncompressed FullKV reference.

These plots complement the fixed-budget tables by showing whether an advantage persists across multiple memory regimes rather than at a single selected budget. They also distinguish methods that improve performance at matched memory from those that reduce memory at comparable performance.

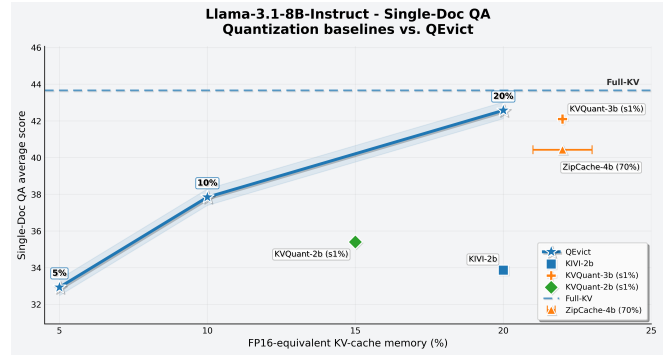
E RULER Evaluation Details

E.1 Dataset and Evaluation Protocol

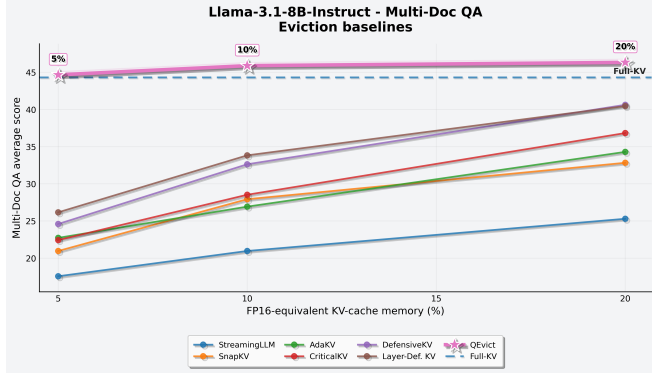
RULER (Hsieh et al. 2024) provides a controlled evaluation of long-context retrieval, aggregation, and reasoning by varying the location, multiplicity, and dependency structure of relevant evidence. We evaluate Llama-3.1-8B-Instruct at a 32K context length under a 20% KV-cache budget. The complete results for this setting are reported in the main paper in Table 2. This section provides additional details about the benchmark and task composition. For RULER, we use a protected recent region of



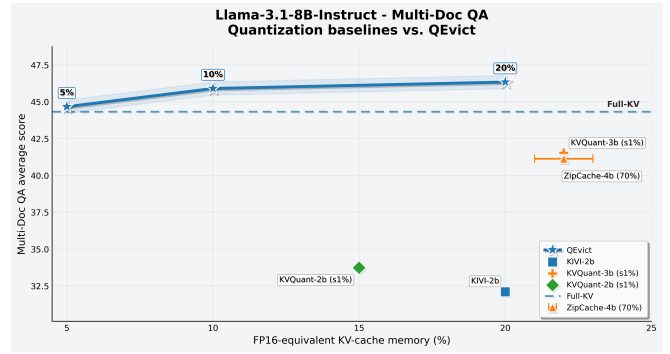
(a) Single-Document QA: eviction.



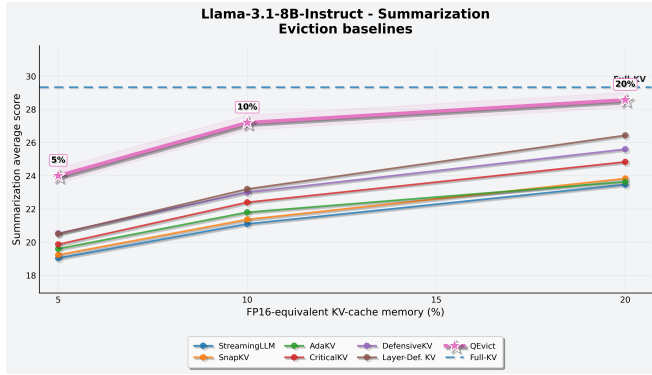
(b) Single-Document QA: quantization.



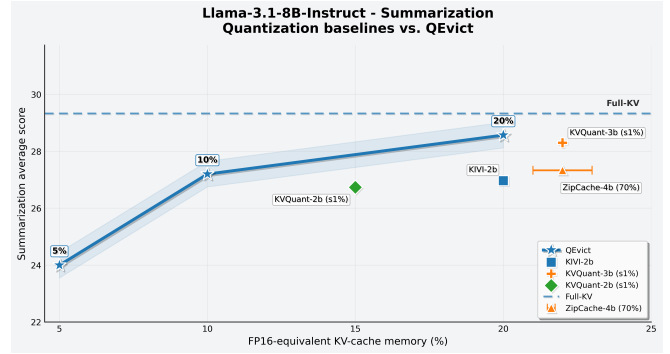
(c) Multi-Document QA: eviction.



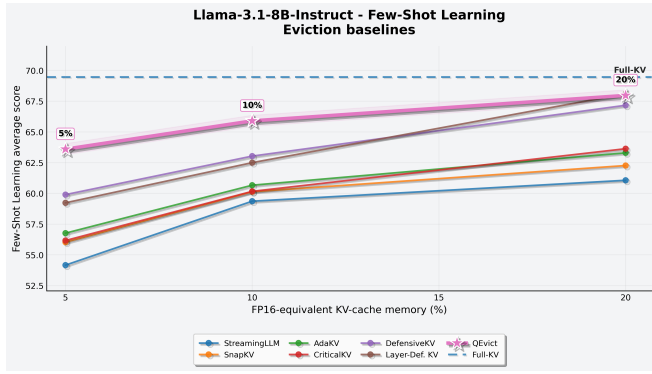
(d) Multi-Document QA: quantization.



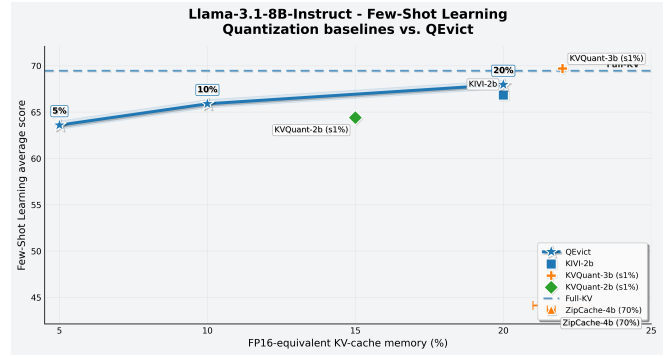
(e) Summarization: eviction.



(f) Summarization: quantization.

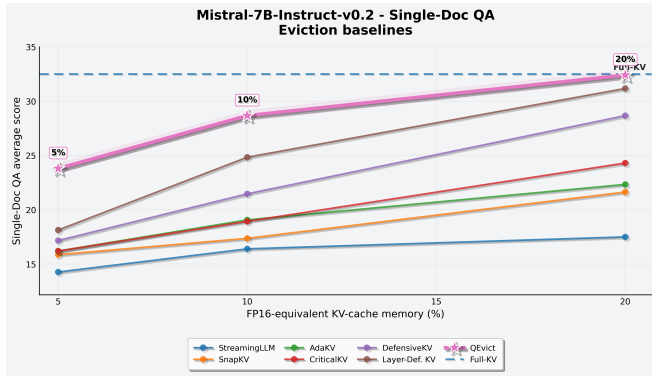


(g) Few-Shot Learning: eviction.

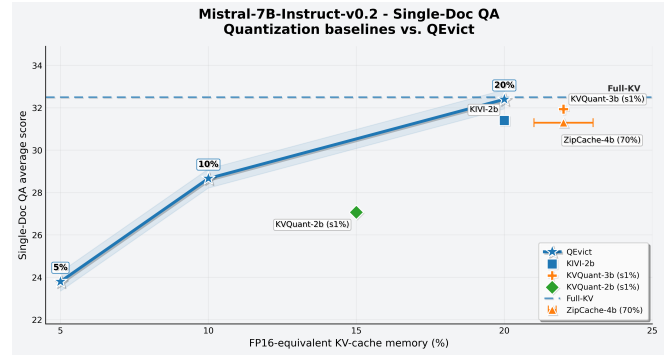


(h) Few-Shot Learning: quantization.

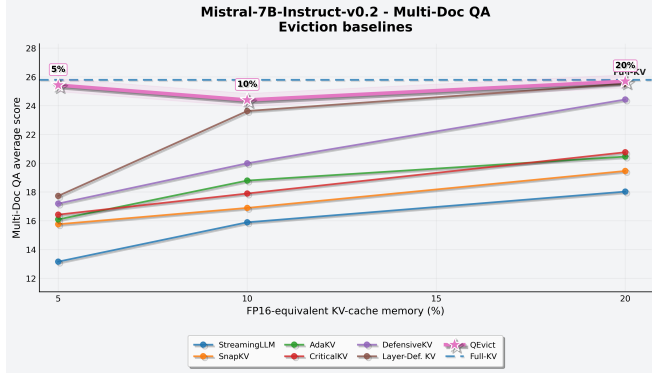
Figure 4: **LongBench performance–memory Pareto analysis for Llama-3.1-8B-Instruct.** Rows correspond to Single-Document QA, Multi-Document QA, Summarization, and Few-Shot Learning. The left and right columns compare `QEvict` with eviction and quantization baselines, respectively. Dashed horizontal lines denote FullKV performance.



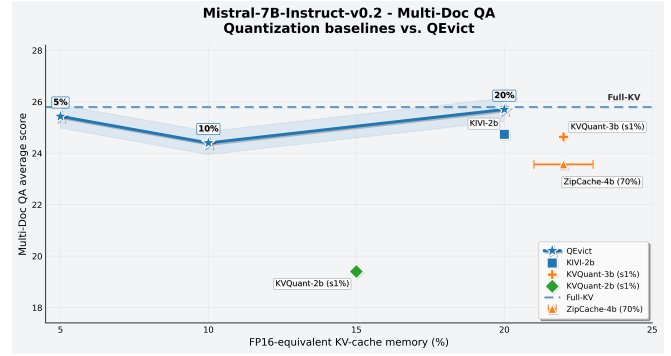
(a) Single-Document QA: eviction.



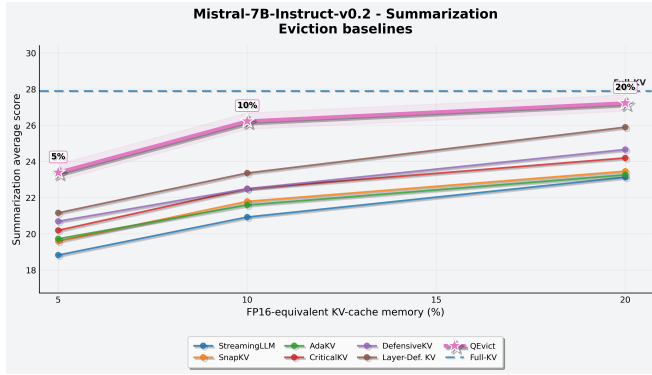
(b) Single-Document QA: quantization.



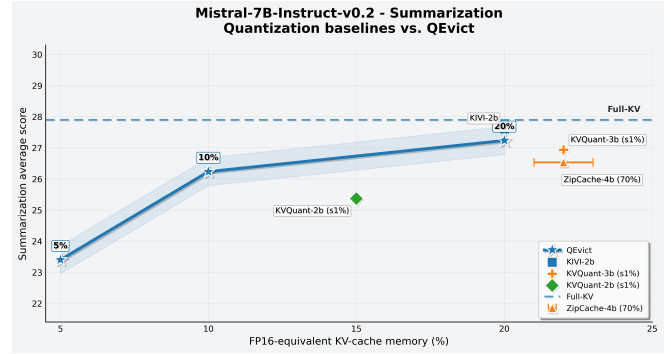
(c) Multi-Document QA: eviction.



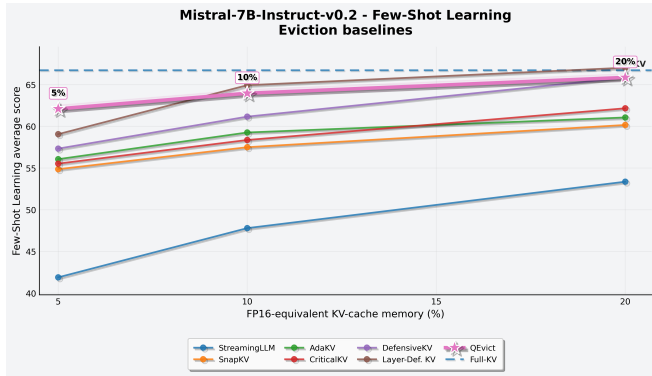
(d) Multi-Document QA: quantization.



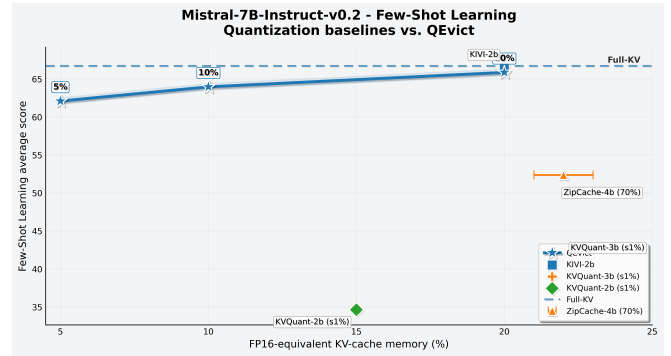
(e) Summarization: eviction.



(f) Summarization: quantization.

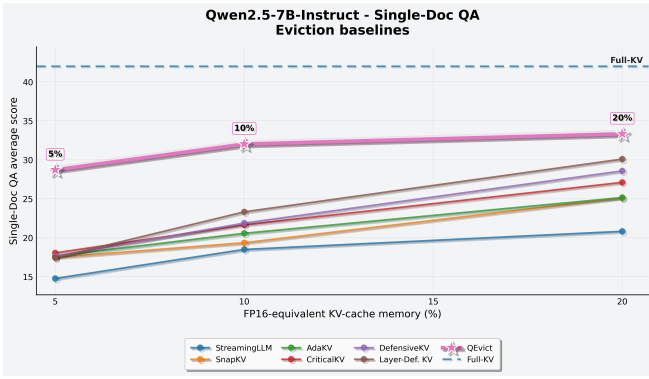


(g) Few-Shot Learning: eviction.

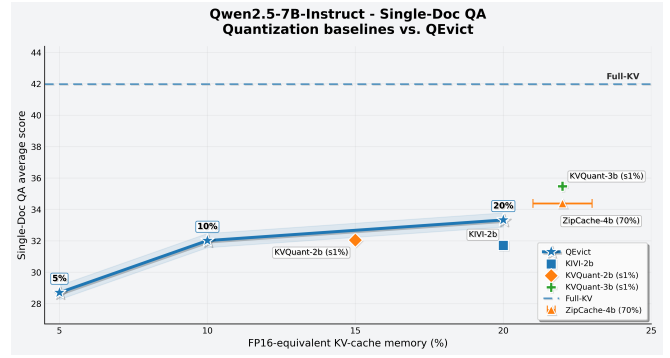


(h) Few-Shot Learning: quantization.

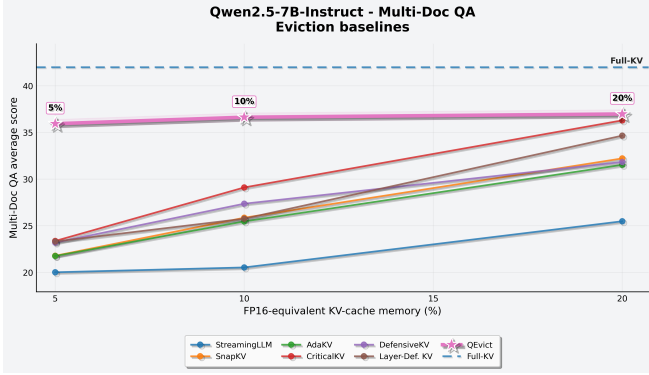
Figure 5: **LongBench performance–memory Pareto analysis for Mistral-7B-Instruct-v0.2.** Rows correspond to Single-Document QA, Multi-Document QA, Summarization, and Few-Shot Learning. The left and right columns compare QEvict with eviction and quantization baselines, respectively. Dashed horizontal lines denote FullKV performance.



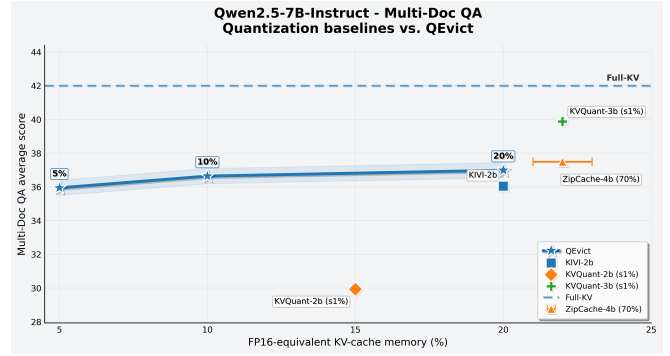
(a) Single-Document QA: eviction.



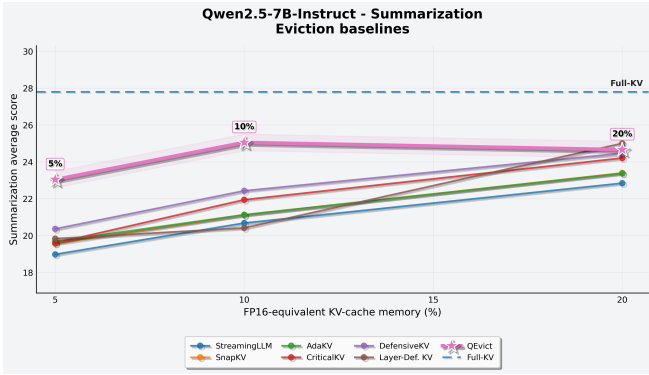
(b) Single-Document QA: quantization.



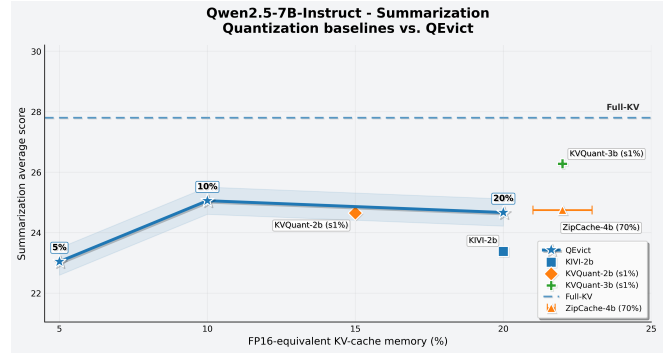
(c) Multi-Document QA: eviction.



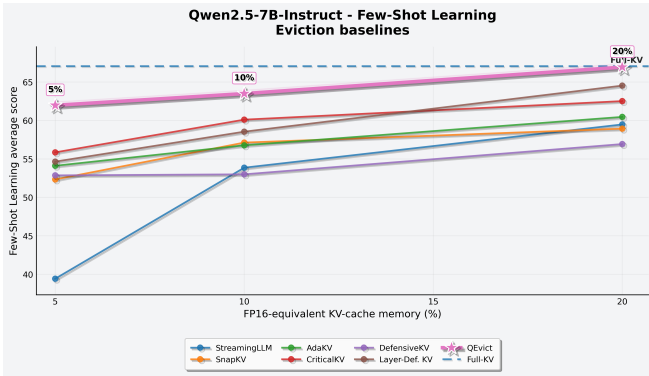
(d) Multi-Document QA: quantization.



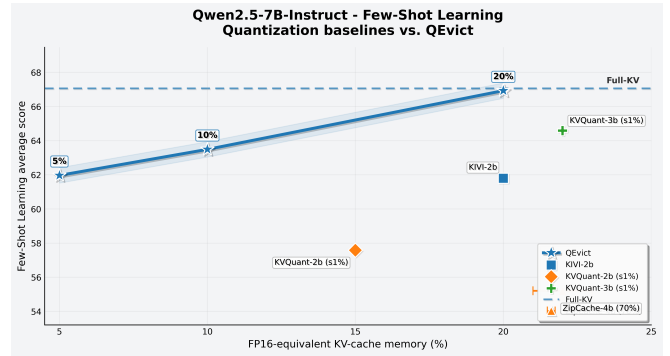
(e) Summarization: eviction.



(f) Summarization: quantization.



(g) Few-Shot Learning: eviction.



(h) Few-Shot Learning: quantization.

Figure 6: LongBench performance–memory Pareto analysis for Qwen2.5-7B-Instruct. Rows correspond to Single-Document QA, Multi-Document QA, Summarization, and Few-Shot Learning. The left and right columns compare QEvict with eviction and quantization baselines, respectively. Dashed horizontal lines denote FullKV performance.

Table 7: **RULER task families used in our evaluation.**

Task Family	Tasks	Evaluation Focus
Aggregation	CWE, FWE	Aggregating frequency information distributed throughout the context.
Needle-in-a-Haystack	S-1–S-3, MK-1–MK-3, MQ, MV	Retrieving sparse evidence under varying numbers of keys, values, and queries.
Question Answering	QA-1, QA-2	Retrieving and combining evidence from relevant passages.
Variable Tracking	VT	Preserving and following dependent variable assignments across the sequence.

64 tokens.

RULER contains 13 tasks spanning four complementary capabilities, summarized in Table 7. These task families expose complementary failure modes of KV-cache compression. Needle-in-a-haystack tasks test whether sparse evidence remains accessible when its relevance is revealed only at query time. Aggregation requires retaining information distributed across the sequence, while variable tracking evaluates the preservation of dependent intermediate states. The question-answering tasks provide a more naturalistic assessment of long-range evidence use.

F GSM8K Evaluation

F.1 Evaluation Protocol

GSM8K (Mirzadeh et al. 2024) evaluates KV-cache compression during autoregressive multi-step reasoning, complementing long-context benchmarks that primarily test retrieval from the input sequence. Successful generation requires maintaining intermediate decoding states that may become relevant again during later calculations or final-answer construction.

We evaluate Llama-3.1-8B-Instruct (Dubey et al. 2024), Qwen2.5-7B-Instruct (Yang et al. 2025), and Mistral-7B-Instruct-v0.2 (Jiang et al. 2023) across multiple measured KV-memory budgets. All methods use the same prompts, greedy decoding procedure, generation limits, and answer-extraction protocol. Performance is reported as exact-match accuracy, and memory is measured relative to the FullKV cache. Eviction methods are compared at matched memory budgets, while quantization methods are reported at their measured operating points following Appendix B.3. Because GSM8K sequences are relatively short, we reserve 25% of the total persistent KV-cache budget for recent tokens.

Fig 3 in the main paper presents the accuracy–memory curves for all three models. Tables 8,9,10 report the corresponding numerical results for every evaluated operating point.

Table 8: **GSM8K performance on Llama-3.1-8B-Instruct.** The left subtable compares KV-cache eviction methods at different KV-cache budgets. The right subtable compares KV-cache quantization methods at approximately matched memory.

(a) KV-cache eviction						(b) KV-cache quantization		
Method	KV-Cache Budget					Method	Equivalent KV Memory	Score
	10%	20%	40%	60%	80%	Full-KV	100%	86.93
Full-KV	86.93	86.93	86.93	86.93	86.93	QEvict	20%	79.09
DefensiveKV	3.11	11.52	76.04	81.65	84.76	KIVI-2b	~20%	79.98
Layer-Def. KV	3.18	<u>31.16</u>	<u>82.56</u>	<u>84.08</u>	84.23	ZipCache-4b	~22%	84.46
CriticalKV	<u>3.26</u>	6.97	31.24	59.14	81.88	KVQuant-2b	~15%	76.61
SnapKV	1.90	4.09	21.61	50.95	76.95	KVQuant-3b	~22%	<u>83.18</u>
AdaKV	2.20	5.16	23.97	52.77	72.82			
StreamingLLM	1.97	3.56	66.79	82.87	84.46			
QEvict	11.45	79.09	85.44	85.97	85.32			

G Ablation Studies

We analyse the sensitivity of QEvict to its three principal design parameters: routing window size, quantized-tier allocation, and recoverable precision. All experiments use Llama-3.1-8B-Instruct under a 5% KV-cache budget. To keep the ablation cost manageable while covering the major LongBench capabilities, we select one task from each of its four task families: NarrativeQA

Table 9: **GSM8K performance on Mistral-7B-Instruct-v0.2.** The left subtable compares KV-cache eviction methods at different KV-cache budgets. The right subtable compares KV-cache quantization methods at approximately matched memory footprints. Full-KV is excluded from the best and second-best rankings.

(a) KV-cache eviction						(b) KV-cache quantization		
Method	KV-Cache Budget					Method	Equivalent KV Memory	Score
	10%	20%	40%	60%	80%			
Full-KV	46.17	46.17	46.17	46.17	46.17	Full-KV	100%	46.17
DefensiveKV	<u>2.35</u>	10.92	<u>39.88</u>	42.68	43.87	QEvict	20%	40.12
Layer-Def. KV	1.90	<u>20.09</u>	39.80	43.79	<u>43.72</u>	KIVI-2b	~20%	<u>40.94</u>
CriticalKV	2.05	5.23	11.22	21.99	36.16	ZipCache-4b	~22%	40.64
SnapKV	1.52	2.73	7.88	18.50	32.83	KVQuant-2b	~15%	38.12
AdaKV	1.52	3.41	10.77	19.94	30.40	KVQuant-3b	~22%	41.79
StreamingLLM	1.59	1.97	29.57	40.26	42.23			
QEvict	18.92	40.12	43.67	<u>43.58</u>	43.44			

Table 10: **GSM8K performance on Qwen2.5-7B-Instruc.** The left subtable compares KV-cache eviction methods at different KV-cache budgets. The right subtable compares KV-cache quantization methods at approximately matched memory footprints. Full-KV is excluded from the best and second-best rankings.

(a) KV-cache eviction						(b) KV-cache quantization		
Method	KV-Cache Budget					Method	Equivalent KV Memory	Score
	10%	20%	40%	60%	80%			
Full-KV	90.78	90.78	90.78	90.78	90.78	Full-KV	100%	90.78
DefensiveKV	2.20	<u>6.37</u>	<u>75.21</u>	81.73	82.87	QEvict	20%	79.42
Layer-Def. KV	2.20	3.71	59.14	80.67	82.18	KIVI-2b	~20%	84.61
CriticalKV	1.59	3.94	10.39	30.78	75.66	ZipCache-4b	~22%	89.99
SnapKV	1.74	2.58	6.44	20.24	51.63	KVQuant-2b	~15%	77.30
AdaKV	2.27	3.56	13.42	38.74	74.37	KVQuant-3b	~22%	<u>89.76</u>
StreamingLLM	2.35	2.50	68.01	84.84	89.84			
QEvict	13.39	79.42	80.51	<u>84.08</u>	<u>87.34</u>			

for single-document question answering, MuSiQue for multi-document question answering, QMSum for summarization, and TriviaQA for few-shot learning. We vary one parameter at a time while keeping the remaining settings fixed to the default configuration ($\Omega = 8$, $q = 0.70$, INT2).

G.1 Window Granularity

We vary the routing window size over $\Omega \in \{4, 8, 16, 32\}$, while fixing $q = 0.70$ and using INT2 for the recoverable tier. Smaller windows enable finer-grained allocation, whereas larger windows provide smoother and more stable importance estimates at the cost of coarser selection. This ablation measures how routing granularity affects performance across the four representative tasks.

G.2 Quantized-Tier Allocation

We vary the fraction of the historical budget assigned to the recoverable tier over

$$q \in \{0.10, 0.30, 0.50, 0.70, 0.90\},$$

while fixing $\Omega = 8$ and using INT2 quantization. Increasing q expands low-bit historical coverage but reduces the capacity of the full-precision historical tier. This sweep isolates the trade-off between precise retention and broader recoverable context.

Table 11: **Effect of routing-window size on LongBench.** Results use Llama-3.1-8B-Instruct under a 5% KV-cache budget with $q = 0.70$ and INT2 quantization. One representative task is selected from each LongBench family. The best and second-best results in each column are shown in bold and underlined, respectively. The highlighted row denotes the default configuration used in the main experiments.

Ω	Nar.QA	MuSiQue	QMSum	Tri.QA	Avg.
4	28.17	30.05	23.75	<u>89.26</u>	42.81
8	<u>27.30</u>	32.50	23.90	89.60	43.33
16	26.78	<u>31.93</u>	23.66	89.17	<u>42.89</u>
32	26.92	30.91	<u>23.78</u>	87.67	42.32

Table 12: **Effect of quantized-tier allocation on LongBench.** Results use Llama-3.1-8B-Instruct under a 5% KV-cache budget with $\Omega = 8$ and INT2 quantization. One representative task is selected from each LongBench family. The best and second-best results in each column are shown in bold and underlined, respectively. The highlighted row denotes the default configuration used in the main experiments.

q	Nar.QA	MuSiQue	QMSum	Tri.QA	Avg.
0.10	26.08	29.76	23.78	87.87	41.87
0.30	<u>27.20</u>	31.08	23.47	88.49	42.56
0.50	27.03	30.67	23.73	89.39	42.71
0.70	27.30	32.50	23.90	89.60	43.33
0.90	27.02	32.58	<u>23.85</u>	89.66	<u>43.28</u>

G.3 Quantization Precision

We compare INT2 and INT4 recoverable representations while fixing $\Omega = 8$ and $q = 0.70$. Under the same total KV-cache budget, INT2 retains more historical windows, whereas INT4 represents fewer windows with lower quantization error. This comparison evaluates whether broader recoverable coverage or higher per-window precision is more beneficial.

Table 13: **Effect of recoverable-tier precision on LongBench.** Results use Llama-3.1-8B-Instruct under a 5% KV-cache budget with $\Omega = 8$ and $q = 0.70$. One representative task is selected from each LongBench family. The best and second-best results in each column are shown in bold and underlined, respectively. The highlighted row denotes the default configuration used in the main experiments.

Precision	Nar.QA	MuSiQue	QMSum	Tri.QA	Avg.
INT2	<u>27.30</u>	32.50	23.90	89.60	43.33
INT4	27.75	<u>30.09</u>	<u>23.88</u>	<u>88.94</u>	<u>42.67</u>

H Efficiency and Systems Analysis

H.1 Measurement Protocol

All efficiency measurements use identical hardware, model precision, batch size, sequence lengths, and execution backend within each comparison. We run warm-up iterations before measurement to initialize kernels and memory pools, synchronize the GPU around timed regions, and reset peak-memory statistics immediately before each run.

We report time to first token (TTFT), time per output token (TPOT), aggregate decoding throughput, and peak GPU memory. For a run producing N_{out} tokens, with timestamps t_{start} , t_{first} , and t_{end} , we compute

$$\text{TPOT} = \frac{t_{\text{end}} - t_{\text{first}}}{N_{\text{out}} - 1}, \quad \text{Throughput} = \frac{N_{\text{out}}}{t_{\text{end}} - t_{\text{start}}}. \quad (10)$$

For batched inference, throughput is aggregated across all generated tokens in the batch. Peak GPU memory includes model parameters, persistent KV-cache storage, attention workspaces, and temporary routing buffers. The complete end-to-end results are reported in Table 4 of the main paper.

H.2 Runtime Components

Compared with FullKV, QEvict introduces periodic score aggregation, window ranking, tier assignment, and cache migration. First-time demotion also creates the persistent low-bit representation, which is reused during subsequent tier transitions. Since routing occurs every Ω generated tokens, its amortized per-token cost is

$$T_{\text{route/token}} = \frac{T_{\text{score}} + T_{\text{rank}} + T_{\text{migrate}}}{\Omega}. \quad (11)$$

At each decoding step, active quantized windows are dequantized and assembled with the full-precision cache before attention. Routing events additionally materialize attention probabilities for cumulative scoring. These operations explain the backend-dependent behaviour observed in the main paper: reducing the attended full-precision cache benefits eager execution, while the current FlashAttention-2 path incurs overhead from dequantization, cache reconstruction, and selective score materialization. Fusing these operations with mixed-precision attention remains the primary systems optimization opportunity.

H.3 Hardware Settings

All experiments were run on a node with 8 NVIDIA A100 80GB GPUs, with a subset additionally run on 1 NVIDIA GB10 128GB GPU. Larger evaluation sweeps were parallelized by sharding the dataset round-robin across GPUs, with one full model copy per GPU. The software stack was PyTorch 2.6.0 (CUDA 12.4), Transformers 4.47.1 and flash-attn 2.8.3.

I Additional Observation Details

This section provides the diagnostic protocol, formal metric definitions, and supporting results for the observations in Section 3.

I.1 Diagnostic Protocol

We collect reference attention traces from an uncompressed FullKV execution because attention assigned to a discarded state cannot be recovered from the compressed execution itself. Unless stated otherwise, diagnostics are computed independently for each layer and attention head and then aggregated across eligible routing events and traces.

The analysis uses Llama-3.1-8B-Instruct (Dubey et al. 2024) with a 512-token prefill, 256 generated tokens, a measured 20% KV-cache budget, and five protected sink tokens. We evaluate two input articles across all 32 transformer layers, yielding 64 layer-level traces. Routing begins at decoding step zero and is repeated every Ω generated tokens. We consider the following conditions:

- **R0**: uncompressed FullKV reference;
- **R1**: token-level eviction with $\Omega = 1$;
- **R2**: two-tier window eviction with $\Omega = 8$;
- **R3**: three-tier routing with $\Omega = 8$, $K_f = 7$, and $K_q = 28$;
- **R4**: three-tier routing with $\Omega = 32$, $K_f = 1$, and $K_q = 11$.

R2 and R3 use the same window size and measured byte budget, isolating the effect of replacing part of the full-precision allocation with a recoverable INT2 tier. R1, R3, and R4 are complete operating points with different routing granularities.

Let $A_t^{\ell,h}(i)$ denote the FullKV attention probability assigned at decoding step t , layer ℓ , and head h to historical token i . The cumulative reference score of window w is

$$S_t^{\ell,h}(w) = \sum_{\tau \leq t} \sum_{i \in w} A_{\tau}^{\ell,h}(i). \quad (12)$$

I.2 Observation I: Future Missed Mass and Selection Churn

Future Missed Mass (FMM). At routing event r , let $\mathcal{E}_r^{\ell,h}$ denote the historical positions made inaccessible by the evaluated policy. We define FMM over a future horizon H as

$$\text{FMM}_H^{\ell,h}(r) = \frac{\sum_{\tau=r+1}^{\min(r+H,T)} \sum_{\substack{i \in \mathcal{E}_r^{\ell,h} \\ i \leq r}} A_{\tau}^{\ell,h}(i)}{\sum_{\tau=r+1}^{\min(r+H,T)} \sum_{i \leq r} A_{\tau}^{\ell,h}(i)}. \quad (13)$$

Only states that already existed at routing event r are included. We use $H = 32$. Lower FMM indicates that the routing decision permanently discards less information required by subsequent queries.

Selection Churn. Let $\mathcal{R}_r^{\ell,h}$ denote the historical positions retained after routing event r . Window selections are expanded to token positions before comparison. Selection Churn is the Jaccard distance between retained sets at consecutive routing events:

$$\text{Churn}^{\ell,h}(r) = 1 - \frac{|\mathcal{R}_r^{\ell,h} \cap \mathcal{R}_{r+1}^{\ell,h}|}{|\mathcal{R}_r^{\ell,h} \cup \mathcal{R}_{r+1}^{\ell,h}|}. \quad (14)$$

The protected recent region is excluded because its deterministic movement reflects the sliding local policy rather than instability in historical selection.

Table 14: **Future Missed Mass and historical-set churn.** FMM is averaged over eligible routing events with $H = 32$. Churn values match those reported in Section 3.

Condition	FMM ↓	Churn/Route ↓
R1: $\Omega = 1$	51.73%	0.0170
R3: $\Omega = 8$	46.38%	0.0012
R4: $\Omega = 32$	47.38%	0.0180
R3: FP tier only	60.19%	–

Table 14 shows that R3 reduces average FMM relative to token-level R1 while lowering historical-set churn by more than an order of magnitude. R4 also reduces FMM relative to R1, but its coarser routing granularity does not improve churn in this measurement. Window aggregation is therefore beneficial, although excessively large windows can sacrifice allocation granularity and need not yield more stable assignments.

The R3 full-precision-only ablation treats quantized windows as inaccessible without reallocating their bytes. Its substantially higher FMM isolates the future attention preserved by the recoverable tier. This is a mechanistic ablation rather than an iso-memory comparison.

I.3 Observation II: Tier-Mass Distribution and Quantized-Score Agreement

For tier $z \in \{\text{FULL}, \text{QUANTIZED}, \text{EVICTED}, \text{LOCAL}\}$, we define its cumulative attention-mass share as

$$\mu_t(z) = \frac{\sum_{w: z_t(w)=z} S_t(w)}{\sum_{w \in \mathcal{W}_t} S_t(w)}. \quad (15)$$

Under the byte-matched R2–R3 comparison, R2 assigns 33.3% of the reference attention mass to full-precision windows and 64.8% to evicted windows. R3 assigns 19.7% to full precision and preserves another 42.6% in INT2, reducing the evicted share to 35.7%. The protected local region contributes 1.9% in both conditions. The recoverable tier therefore reduces the reference attention mass assigned to permanent eviction by 29.1 percentage points.

To determine whether quantized windows remain suitable for subsequent ranking, let $\mathbf{s}_r^Q$ denote their scores under INT2 execution and $\mathbf{s}_r^{\text{Full}}$ the corresponding FullKV scores. We define Quantized-Score Agreement as

$$\text{QSA}(r) = \frac{\langle \mathbf{s}_r^Q, \mathbf{s}_r^{\text{Full}} \rangle}{\|\mathbf{s}_r^Q\|_2 \|\mathbf{s}_r^{\text{Full}}\|_2}. \quad (16)$$

We additionally measure the ratio between the total attention mass assigned to these windows under quantized and full-precision execution:

$$R_Q(r) = \frac{\sum_{w \in \mathcal{Q}_r} S_r^Q(w)}{\sum_{w \in \mathcal{Q}_r} S_r^{\text{Full}}(w)}. \quad (17)$$

The high cosine agreement in Table 15 shows that INT2 largely preserves the relative ordering of quantized windows. The lower mass ratio indicates attenuation in attention magnitude, so low-bit execution is not numerically identical to FullKV. Nevertheless, the ranking signal remains sufficiently stable for continued scoring and subsequent promotion.

Table 15: Agreement between INT2 and FullKV scores over R3 quantized windows.

Statistic	Value
Mean cosine agreement	0.9824
First / last routing event	1.0000/0.9676
Per-layer minimum / maximum	0.9503/0.9958
Per-head minimum / maximum	0.7874/0.9995
Mean attention-mass ratio	0.8660

I.4 Observation III: Global LIR

Let $X_{r,w} \in \{0, 1\}$ indicate whether window w belongs to a selected full-precision set at routing event r . We analyse two selections:

- **Oracle**: the top- K_f windows according to the R0 FullKV ranking;
- **Policy FP**: the full-precision tier selected by R3 or R4.

An inactive episode is a maximal sequence of routing events for which $X_{r,w} = 0$. After m consecutive inactive events, the episode becomes eligible. It is rescued if the window re-enters the selected set at any later routing event before generation terminates. We define

$$\text{GlobalLIR}(m) = \frac{\#\{\text{eligible inactive episodes that later re-enter}\}}{\#\{\text{eligible inactive episodes}\}}. \quad (18)$$

Each maximal inactive episode contributes once. Episodes that do not return before generation terminates remain in the denominator and are counted as not rescued. We use $m = 3$ by default and verify the same trend for $m \in \{1, 2, 4, 8\}$.

Table 16: **Global LIR for oracle and policy full-precision selections.** Confidence intervals are 95%. Median time-to-return is measured in routing events.

Run	Selection	Global LIR	Rescued	Median TTR
R3	Oracle	0.98% [0.92%, 1.03%]	51	12
R3	Policy FP	6.18% [6.09%, 6.27%]	341	8
R4	Oracle	0.09% [0.00%, 0.17%]	1	4
R4	Policy FP	2.59% [1.90%, 3.27%]	30	2

The low oracle LIR in Table 16 confirms that the true top-ranked region is largely persistent. Under R3, however, the deployed full-precision tier exhibits a 6.18% return rate, approximately $6.3\times$ the oracle rate. Approximate scoring and constrained allocation therefore produce substantially greater re-entry pressure than the oracle ranking alone suggests.

The median policy return times are eight routing events for R3 and two for R4. Given routing intervals of eight and 32 tokens, respectively, both correspond to approximately 64 decoding steps. Relevance revival can therefore occur well beyond the immediately following routing event.

We complement Global LIR with the lagged transition probability

$$P_{ab}^{(\Delta)} = \Pr[X_{r+\Delta,w} = b \mid X_{r,w} = a], \quad a, b \in \{0, 1\}. \quad (19)$$

For the R3 policy, $P_{01}^{(\Delta)}$ increases from 0.31% at $\Delta = 1$ to 2.39% at $\Delta = 8$, while $P_{10}^{(\Delta)}$ increases from 3.05% to 22.12%. At $\Delta = 8$, departures from the full-precision tier are approximately $9.3\times$ more frequent than returns.

These dynamics are asymmetric. Most oracle-important windows remain active, many policy-selected windows subsequently decline, and a smaller but systematic subset eventually returns. This behaviour motivates frequent demotion together with recoverable low-bit storage that preserves the possibility of later promotion.

J Compatibility with Alternative Ranking Functions

The three-tier hierarchy in QEvict requires only an ordering over candidate windows and is therefore not tied to cumulative attention as its importance estimator. To evaluate this modularity, we replace the default ranking function with three representative alternatives while leaving the cache hierarchy, byte allocation, and migration policy unchanged.

All experiments use Llama-3.1-8B-Instruct on LongBench under a 5% KV-cache budget with $\Omega = 8$, $q = 0.70$, and an INT2 recoverable tier. The evaluation covers the 12 tasks used in the main study together with PassageCount and PassageRetrieval-en. We compare the following scoring functions:

Table 17: **Compatibility of QEvict with alternative window-ranking functions on LongBench.** All results use Llama-3.1-8B-Instruct under a 5% KV-cache budget with $\Omega = 8$, $q = 0.70$, and INT2 quantization. Only the ranking function is changed. The average is computed over all 14 tasks. Best and second-best distinct results are shown in bold and underlined, respectively.

Ranking function	Single-Document QA			Multi-Document QA			Summarization			Few-Shot Learning			Synthetic		Avg.
	Nar.QA	Qasper	MF-en	Hot.QA	2Wi.QA	MuSiQue	Gov.Re.	QMSum	M.News	TREC	Tri.QA	SAMSum	PCount	PR-en	
Cumulative attention	27.30	28.00	<u>43.50</u>	<u>56.40</u>	45.10	<u>32.50</u>	<u>28.80</u>	<u>23.90</u>	19.30	63.00	<u>89.60</u>	38.20	<u>8.67</u>	<u>92.00</u>	<u>42.59</u>
Exp. decay, $\lambda = 0.99$	27.34	<u>28.03</u>	43.54	56.47	<u>45.19</u>	32.59	29.11	24.13	19.20	63.00	89.66	38.21	<u>8.67</u>	<u>92.00</u>	42.65
Attention p -norm, $p = 3$	29.64	28.80	38.55	54.54	46.51	30.98	26.49	23.84	<u>19.27</u>	<u>61.00</u>	87.47	40.03	9.00	96.00	42.29
Key L_2 -norm	<u>28.73</u>	24.91	40.02	54.12	43.56	30.02	27.34	22.52	18.57	<u>61.00</u>	87.31	<u>39.79</u>	9.00	<u>92.00</u>	41.35

- **Cumulative attention**, the default estimator used by QEvict.
- **Exponentially decayed attention**, which discounts older observations and places greater weight on recent queries. This variant is motivated by historical-attention and persistence-based cache selection (Zhang et al. 2023; Liu et al. 2023).
- **Attention p -norm**, which replaces additive aggregation with a peak-sensitive p -norm over observed window scores. We use $p = 3$. This is a power-norm generalization of the attention-based ranking signals used by method such SnapKV (Li et al. 2024).
- **Key-vector norm**, an attention-free estimator based on the L_2 -norm of cached keys. Following Devoto et al. (2024), lower key norms receive higher retention priority.

Let $u_t^\ell(w)$ denote the attention received by window w since the previous routing event:

$$u_t^\ell(w) = \frac{1}{H_q} \sum_{h=1}^{H_q} \sum_{\tau=t-\Omega+1}^t \sum_{i \in w} a_{\tau}^{\ell,h}(i). \quad (20)$$

The cumulative and exponentially decayed scores are

$$S_{t,\text{cum}}^\ell(w) = S_{t-\Omega,\text{cum}}^\ell(w) + u_t^\ell(w), \quad (21)$$

$$S_{t,\text{decay}}^\ell(w) = \lambda S_{t-\Omega,\text{decay}}^\ell(w) + u_t^\ell(w), \quad \lambda = 0.99. \quad (22)$$

For the p -norm variant, we aggregate the event-level window scores as

$$S_{t,p}^\ell(w) = \left(\sum_{r \leq t} [u_r^\ell(w)]^p \right)^{1/p}, \quad p = 3. \quad (23)$$

The key-norm estimator is defined as

$$S_{\text{key}}^\ell(w) = -\frac{1}{|w|} \sum_{i \in w} \|\mathbf{k}_i^\ell\|_2, \quad (24)$$

where the negative sign converts the low-norm preference into the higher-is-better convention used by the routing algorithm.

K Effect of Q→F Residency Promotion

Objective. We examine whether allowing quantized windows to return to the full execution tier improves the behavior of the R3 three-tier policy. We compare the one-way policy

$$\text{FULL} \rightarrow \text{QUANTIZED} \rightarrow \text{EVICTED}$$

against the bidirectional policy

$$\text{FULL} \leftrightarrow \text{QUANTIZED} \rightarrow \text{EVICTED}.$$

A promoted window is reconstructed from its persistent INT2 representation. Promotion therefore restores full-tier residency and execution handling rather than the original pre-quantization KV values.

Table 18: Effect of Q→F residency promotion for the R3 three-tier policy with window size $\Omega = 8$. QSA and R_Q are computed over the windows currently assigned to the quantized tier. Lower is better for FMM; higher is better for the remaining metrics.

Metric	No promotion	With promotion	Δ
R3 FMM ↓	16.8%	16.1%	−0.7 pp
Quantized-Score Agreement ↑	0.947	0.985	+0.038
FullKV attention mass preserved, R_Q ↑	88.7%	95.5%	+6.8 pp
Global LIR ↑	0.0%	6.6%	+6.6 pp
Recorded Q→F transitions	0	639	+639

Experimental setting. We use Llama-3.1-8B-Instruct (Dubey et al. 2024) with a 256-token prefill, 1024 decoding tokens, a measured 20% persistent KV-cache budget, five protected sink tokens, and the R3 three-tier configuration with window size $\Omega = 8$. The two variants use the same tier capacities, cumulative-attention scoring rule, routing schedule, quantization configuration, and memory budget. They differ only in whether a window already assigned to the quantized tier may later be promoted to the full execution tier. Under the no-promotion variant, quantized windows may subsequently be evicted but cannot return to the full tier.

The current pilot conditions use different source articles. The comparison should therefore be interpreted as directional evidence of the behavior enabled by promotion rather than as a paired estimate of its exact causal effect.

Metrics. We reuse the diagnostics introduced in Appendix I: Future Missed Mass (FMM), Quantized-Score Agreement (QSA), the attention-mass ratio R_Q , and Global LIR. We additionally report the number of observed Q→F transitions as an auxiliary measure of promotion activity.

For this analysis, FMM measures the fraction of future FullKV attention assigned to historical states that have become inaccessible under R3. Lower FMM indicates that fewer future-relevant states are permanently discarded.

QSA measures the cosine agreement between the score vector of the windows currently assigned to the quantized tier and their corresponding FullKV reference scores:

$$\text{QSA}(r) = \frac{\langle \mathbf{s}_r^Q, \mathbf{s}_r^{\text{Full}} \rangle}{\|\mathbf{s}_r^Q\|_2 \|\mathbf{s}_r^{\text{Full}}\|_2}.$$

The attention-mass ratio measures the fraction of the FullKV reference attention mass preserved over the windows currently assigned to the quantized tier:

$$R_Q(r) = \frac{\sum_{w \in Q_r} S_r^Q(w)}{\sum_{w \in Q_r} S_r^{\text{Full}}(w)}.$$

We report R_Q as a percentage. A value of 100% indicates that quantized execution preserves the total FullKV attention mass over the current Q-tier membership.

Global LIR measures the percentage of eligible inactive episodes that later re-enter the selected full-tier set. The transition count records the total number of Q→F movements observed during decoding.

Observation 1: Promotion is associated with lower future missed mass. R3 FMM decreases from 16.8% under one-way demotion to 16.1% when Q→F promotion is enabled. This directional reduction suggests that bidirectional routing can reduce the premature permanent removal of windows whose importance later re-emerges.

Observation 2: Promotion improves policy-level Q-tier score agreement. QSA increases from 0.947 to 0.985, showing that the score vectors of the windows currently remaining in the quantized tier align more closely with their FullKV reference scores under the promotion-enabled policy.

Observation 3: Promotion preserves more FullKV attention mass in the Q tier. The attention-mass ratio R_Q increases from 88.7% to 95.5%, a gain of 6.8 percentage points. Thus, the current Q-tier population under bidirectional routing preserves a larger fraction of its FullKV reference attention mass and moves closer to the ideal value of 100%.

Because promotion changes which windows remain in the quantized tier, the improvements in QSA and R_Q should be interpreted as evidence of better tier composition and policy-level score agreement. They do not imply that promotion reverses the quantization error of a fixed window.

Observation 4: Promotion is actively exercised. Global LIR increases from 0% under the one-way policy to 6.6% when promotion is enabled. The zero value without promotion holds by construction, whereas the nonzero value confirms that previously inactive windows re-enter the full-tier selection under bidirectional routing.

Observation 5: Re-entry is frequent rather than purely hypothetical. The promotion-enabled run records 639 $Q \rightarrow F$ transitions, compared with none under one-way demotion. This shows that promotion is an active component of the routing policy rather than only a permitted but unused state transition.

Overall observation. Taken together, the lower FMM, higher QSA and R_Q , and nonzero re-entry activity provide consistent directional evidence that $Q \rightarrow F$ promotion improves the dynamic allocation of historical windows across the full and quantized tiers.